



Sistemas Embebidos I

Otoño 2026

**Práctica 6: Comunicación serial y
scheduler cooperativo con UART**

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Jonathan Hernández Lazcano

Sábado, 19 de septiembre de 2026

Índice

<i>Práctica 6: Comunicación serial y scheduler cooperativo con UART</i>	2
Introducción	2
Descripción de la práctica	3
Objetivo	4
Marco teórico	5
Desarrollo	5
Materiales	5
Esquemático	6
Desarrollo	8
Resultados	21
Parte 1 — La consola	21
Parte 2 — Límite del sistema	25
Evidencias	34
Conclusiones	36
Uso de IA	37
Referencias	38

Práctica 6: Comunicación serial y scheduler cooperativo con UART

Introducción

Un microcontrolador tiene un solo procesador y, casi siempre, más de una cosa que hacer. En las prácticas anteriores cada programa atendía un único asunto a la vez: medir el periodo de una señal, filtrar el rebote de un botón, generar una base de tiempo. La comunicación serial rompe esa comodidad, porque los datos no llegan cuando el programa está listo para recibirlos, sino cuando el otro extremo decide enviarlos. A 115200 baud cada byte tarda 86.8 microsegundos en llegar, y el periférico solo puede guardar 32 de ellos antes de empezar a perderlos: poco menos de tres milisegundos de margen para todo lo demás que el programa tenga que hacer.

Ese margen es el que convierte un problema de comunicación en un problema de arquitectura. Una espera con `sleep_ms`, una línea impresa con `printf` o incluso un punto de interrupción colocado para investigar bastan para agotarlo, y cuando se agota no aparece ningún error de compilación ni ninguna advertencia: simplemente faltan caracteres. El programa «funciona», nadie se queja y los datos ya no están. Esa pérdida silenciosa es justamente lo que la hace peligrosa, y la única defensa contra ella es instrumentar el programa para que registre lo que pierde.

La solución a la que conduce el problema tiene dos piezas que se complementan. La primera es delegar la recepción a una interrupción, de modo que atender el enlace deje de depender de qué tan ocupado esté el programa principal. La segunda es un buffer circular que desacople los dos ritmos: el del hardware, que entrega bytes cuando llegan, y el del programa, que los consume cuando puede. Sobre esa base se puede construir un bucle de tareas cooperativas en el que ninguna se detenga a esperar, y en el que todas alcancen su turno dentro del presupuesto disponible.

Esta práctica recorre ese camino completo. Primero se construye el sistema —una consola de comandos por UART conviviendo con tres tareas periódicas— y después se lo somete a carga creciente para encontrar por medición, y no por suposición, el punto

en el que empieza a perder datos. Esa segunda mitad resultó la más instructiva, porque el umbral medido no coincidió con el calculado, y entender por qué obligó a revisar qué mide realmente cada uno de los dos números.

Descripción de la práctica

El trabajo se desarrolló sobre dos tarjetas Raspberry Pi Pico 2 W conectadas entre sí por un enlace serial cruzado. La primera, llamada Pico principal, ejecuta el programa bajo prueba y dedica su UART0 por completo al enlace físico. La segunda, la Pico de apoyo, no participa en la lógica del programa y funciona únicamente como puente: traslada hacia su puerto USB lo que recibe por el UART y hacia el UART lo que recibe por el USB. Esta separación es indispensable, porque los errores de trama solo existen sobre el enlace físico y la consola USB los ocultaría por completo.

La práctica se dividió en dos partes. La primera consistió en construir la consola. Se configuró el UART0 a 115200 baud en formato 8N1 y se reportó la velocidad que el hardware consiguió realmente frente a la nominal. La recepción se resolvió por interrupción, con una rutina que se limita a sacar bytes del periférico y depositarlos en un buffer circular, sin interpretar nada. El eco de los caracteres tecleados se produce desde el bucle principal, y el buffer lleva la cuenta de los bytes que descarta cuando se llena. La transmisión también se encoló, para que imprimir no bloquee al programa. Sobre esa base se montaron tres tareas en un mismo bucle —el parpadeo del LED, la atención de la consola y un registro con marca de tiempo— sin que ninguna utilice `sleep_ms`, y se implementó un intérprete de cuatro comandos que permite consultar los contadores de error y modificar el comportamiento del programa en caliente, sin recompilar.

La segunda parte consistió en encontrar el límite del sistema. Se calculó primero, a partir del tiempo de bit y de la capacidad de la FIFO, cuánto puede durar la vuelta del bucle antes de que se pierda un byte. Después se inyectó un retardo controlado y creciente en la tarea del LED —de 0, 1, 3 y 10 milisegundos— y se aplicó tráfico continuo durante treinta segundos en cada caso, consultando los contadores al final de cada corrida. El umbral medido se contrastó con el calculado y se explicó la diferencia entre ambos. Finalmente se provocó de manera deliberada un modo de falla distinto: se desajustó la velocidad del puente a 57600 mientras la Pico principal permanecía en 115200, para

registrar qué ocurre en pantalla, qué reportan los contadores y por qué ese error pertenece a una categoría diferente de la pérdida por desbordamiento.

Objetivo

Objetivo general. Construir un programa capaz de atender comunicación serial y tareas periódicas de forma simultánea sin pérdida de datos, y determinar por medición el punto en que esa pérdida comienza, explicando a qué mecanismo se debe.

Objetivos específicos.

- Configurar el UART0 a 115200 baud en formato 8N1 y cuantificar el error entre la velocidad nominal y la que el hardware consigue realmente.
- Implementar la recepción por interrupción sobre un buffer circular, manteniendo la rutina de atención reducida a mover bytes y contabilizar errores.
- Desacoplar la transmisión mediante una cola, de modo que emitir texto no detenga al programa principal.
- Organizar tres tareas periódicas en un scheduler cooperativo en el que ninguna se bloquee ni ceda el control con esperas activas.
- Implementar una interfaz de comandos por texto que permita consultar los contadores de diagnóstico y modificar parámetros en tiempo de ejecución.
- Calcular el presupuesto de tiempo disponible para la vuelta del bucle a partir del tiempo de bit y de la capacidad de almacenamiento del periférico.
- Medir experimentalmente el umbral de pérdida inyectando retardo creciente bajo tráfico continuo, y contrastarlo con el valor calculado.
- Distinguir experimentalmente los tres mecanismos de pérdida —error de trama, desbordamiento de la FIFO y descarte del buffer de software— provocando un desajuste de velocidad entre los dos extremos del enlace.

Marco teórico

GPIO y entradas digitales

Los GPIO (*General Purpose Input/Output*) son pines de propósito general que pueden configurarse mediante software como entradas o salidas. Cuando un GPIO se configura como entrada, permite que el microcontrolador lea el estado lógico de un componente

externo, como un botón. Cuando se configura como salida, permite controlar dispositivos como LEDs. En el RP2350, la configuración y manipulación de los GPIO puede realizarse mediante el SDK de Raspberry Pi Pico. Los GPIO pueden representar estados digitales mediante niveles alto y bajo, asociados normalmente con valores lógicos de 1 y 0.

Para utilizar un botón como entrada digital es necesario establecer una referencia eléctrica que determine su estado cuando no está siendo presionado. Una forma de realizarlo es utilizar una resistencia *pull-up*. En este caso, la entrada se mantiene normalmente en nivel lógico alto y, al presionar el botón, la conexión con tierra provoca que la entrada pase a nivel bajo. Esto permite interpretar una pulsación mediante la lectura del GPIO.

Operaciones bitwise y máscaras

Los registros utilizados para controlar los GPIO pueden manejar varios pines de manera simultánea mediante bits. Cada bit representa un GPIO específico; por ejemplo, el bit 2 corresponde al GPIO 2 y el bit 25 al GPIO 25. Para seleccionar un GPIO determinado se puede utilizar el operador de desplazamiento de bits $\ll$. La expresión $1u \ll PIN$ genera una máscara en la que solamente se encuentra activo el bit correspondiente al número del pin.

Las máscaras también permiten trabajar con varios GPIO de manera simultánea. Mediante el operador OR ($|$) se pueden combinar diferentes máscaras, por ejemplo: $(1u \ll PIN1) | (1u \ll PIN2)$.

De esta forma, una sola máscara puede representar un conjunto de pines. Este principio es utilizado en la extensión de la práctica para controlar dos LEDs independientes mediante una máscara multipin.

Rebote

Un botón mecánico no produce necesariamente una transición digital limpia en el instante en que es presionado. Debido al comportamiento físico de sus contactos, la

señal puede cambiar varias veces entre los niveles alto y bajo durante un periodo corto antes de estabilizarse. Este fenómeno se conoce como rebote mecánico.

El rebote puede provocar que un programa interprete una sola pulsación física como varias pulsaciones. Por ejemplo, si cada transición detectada produce un cambio en el estado de un LED, una sola pulsación podría provocar múltiples cambios no deseados.

Para solucionar este problema se utiliza una técnica de antirrebote (debounce). El material de la práctica presenta tres estrategias principales: un retardo fijo, una ventana o integrador de lecturas coherentes y una máquina de estados. El retardo fijo puede utilizar `sleep_ms()` pero mientras el procesador espera no puede realizar otras tareas. Por otra parte, una FSM permite continuar realizando muestreos y solamente confirmar una transición cuando la señal ha permanecido estable durante el tiempo establecido.

Máquina de estados finitos FSM

Una máquina de estados finitos (FSM) es un modelo utilizado para representar sistemas que pueden encontrarse en un conjunto definido de estados y cambiar entre ellos cuando ocurre un determinado evento. Una FSM está compuesta principalmente por estados y transiciones. Los estados representan los diferentes modos en los que puede encontrarse el sistema, mientras que las transiciones indican las condiciones necesarias para pasar de un estado a otro.

Para el antirrebote de un botón, una FSM permite distinguir entre una entrada estable y una transición que todavía puede corresponder al rebote mecánico. El sistema puede comenzar en un estado de botón liberado, pasar a un estado de comprobación cuando detecta una pulsación y confirmar posteriormente la transición si el botón continúa en el mismo estado durante el tiempo de antirrebote. De igual manera, se puede comprobar la liberación del botón antes de permitir que una nueva pulsación sea reconocida.

La implementación mediante FSM utiliza una referencia temporal para determinar cuánto tiempo ha permanecido estable la entrada. Para este tipo de antirrebote, se puede utilizar `get_absolute_time()` en lugar de `sleep_ms()`. Esto permite que el programa

continúe ejecutando otras operaciones mientras espera a que transcurra el intervalo de comprobación.

Este principio es importante cuando se utilizan varios botones. Un programa bloqueado esperando a que termine el antirrebote de un botón podría retrasar la lectura de otro botón. En cambio, de esta manera permite revisar continuamente diferentes entradas y mantener su propio estado y referencia temporal.

UART

La UART (Universal Asynchronous Receiver/Transmitter) es un periférico de comunicación serial que permite transmitir y recibir datos entre dispositivos mediante una comunicación asíncrona. En este tipo de comunicación, los datos se envían de manera secuencial, bit por bit, utilizando una línea de transmisión (TX) y una línea de recepción (RX). La comunicación requiere que ambos dispositivos utilicen la misma configuración de velocidad y formato de los datos. En el material se utiliza como configuración de referencia 115200 baudios, 8N1, donde se emplean 8 bits de datos, sin paridad y un bit de parada.

En la práctica, la UART puede utilizarse para establecer comunicación directa entre dos Raspberry Pi Pico. Una de las placas puede actuar como dispositivo transmisor y la otra como receptor. Para establecer la comunicación, la salida TX de la primera Raspberry se conecta con la entrada RX de la segunda, mientras que la línea TX de la segunda se conecta con la entrada RX de la primera si se requiere comunicación bidireccional.

Desarrollo

Materiales

- 2 tarjetas Raspberry Pi Pico 2 W (microcontrolador RP2350, ARM Cortex-M33): una como *Pico principal*, que ejecuta el programa bajo prueba, y otra como *Pico de apoyo*, que funciona como puente USB–UART.
- 2 protoboards.
- Cables jumper macho-macho para el enlace serial cruzado, la referencia de tierra y el LED.

- 1 LED y su resistencia limitadora, conectados al pin GP16 de la Pico principal.
- 2 cables USB tipo A a micro-USB, uno por tarjeta.
- Computadora con Visual Studio Code y la extensión *Raspberry Pi Pico*.
- Pico SDK 2.3.1 con la cadena de herramientas arm-none-eabi-gcc 13.2, CMake y Ninja.
- Monitor serial integrado de Visual Studio Code, configurado a 115200 baud con terminación de línea CRLF.
- Python 3 con la biblioteca pyserial, para el script generador de tráfico continuo.
- Lucidchart, para la elaboración de los diagramas de flujo.

Esquemático

El montaje utiliza el UART por hardware del RP2350 y no la consola USB, porque los errores de trama solo existen sobre el enlace físico: la consola USB los ocultaría por completo. Por esa razón se necesitan dos tarjetas. La Pico principal ejecuta el programa de la práctica y su UART0 queda dedicado por entero al enlace serial; la Pico de apoyo no participa en la lógica del programa y se limita a trasladar bytes entre su puerto USB y su UART0, de modo que el monitor serial de la computadora se abre contra el puerto USB de la tarjeta de apoyo y nunca contra el de la principal.

El enlace es cruzado: lo que una tarjeta transmite es lo que la otra recibe. Además de las dos líneas de datos es indispensable unir las tierras, porque sin una referencia común los niveles «alto» y «bajo» no significan lo mismo en ambos extremos y la comunicación falla aunque los baudios coincidan. Las conexiones empleadas se resumen en la tabla 1.

Tabla 1. Conexiones entre la Pico principal, la Pico de apoyo y los periféricos externos.

Señal	Pico principal	Pico de apoyo	Función
TX → RX	GP0 (TX)	GP1 (RX)	Datos que la principal envía hacia la computadora
RX ← TX	GP1 (RX)	GP0 (TX)	Datos que la computadora envía hacia la principal
GND	GND	GND	Referencia común de tensión
LED	GP16	—	LED de estado de la tarea periódica
Traza ISR	GP14	—	Pin expuesto que marca la entrada y salida de la ISR
Traza bucle	GP13	—	Pin expuesto que marca la duración de la vuelta del bucle

Señal	Pico principal	Pico de apoyo	Función
USB	—	USB	Enlace con el monitor serial de la computadora

Los pines GP13 y GP14 se declararon como salidas de traza para poder observar con un osciloscopio la duración de la vuelta del bucle y el tiempo de servicio de la interrupción. Se dejaron expuestos y el código los conmuta en los puntos correspondientes, aunque en esta práctica la medición con osciloscopio no llegó a realizarse.

La figura 1 muestra la estructura completa del programa de la Pico principal, separada en sus cuatro bloques funcionales: la configuración inicial, la rutina de atención a la interrupción del UART, el bucle principal con sus tres tareas y el procesador de comandos. La figura 2 muestra el comportamiento mucho más simple de la Pico de apoyo.

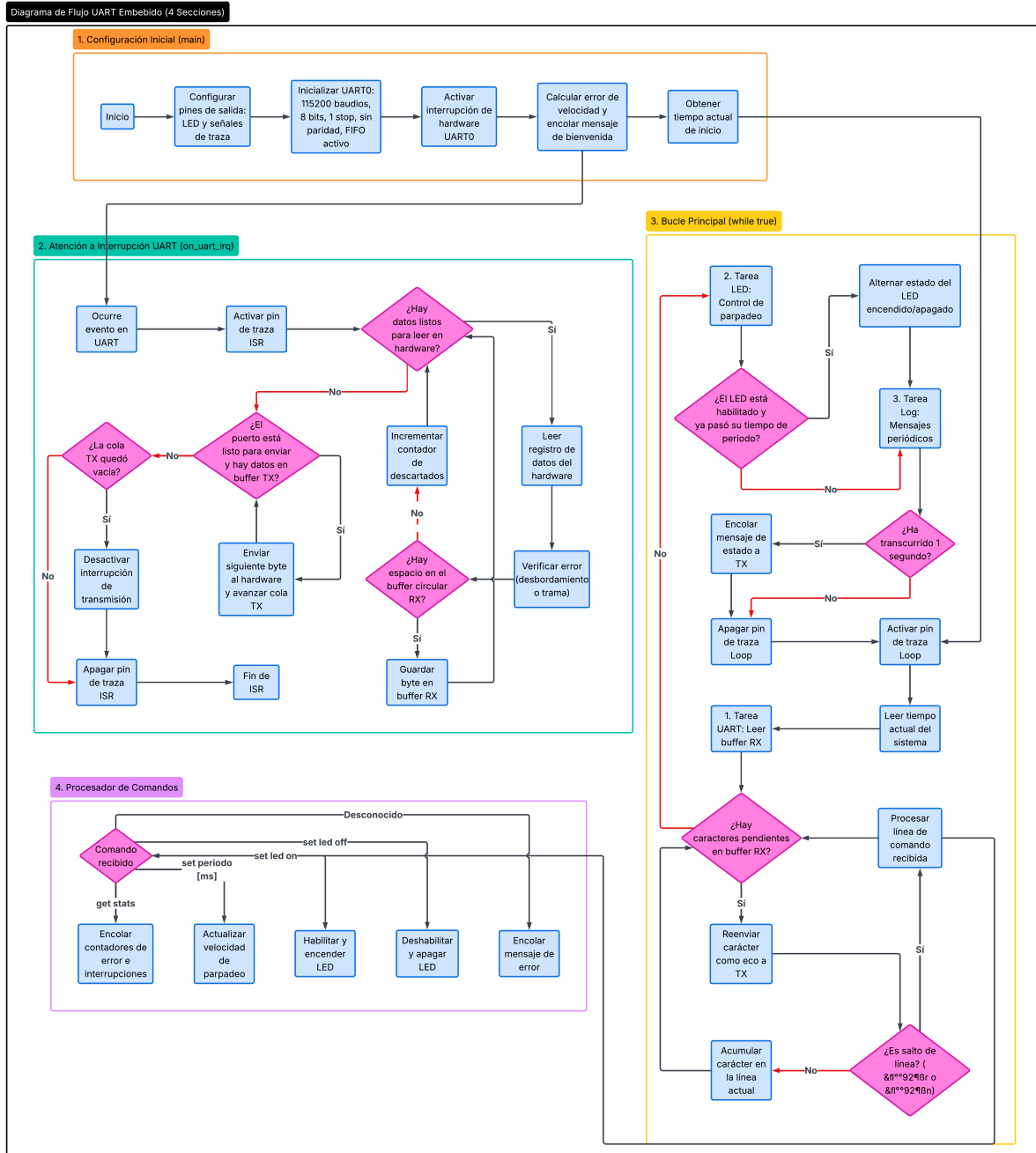


Figura 1. Diagrama de flujo del programa de la Pico principal, dividido en configuración inicial, atención a la interrupción del UART, bucle principal y procesador de comandos.

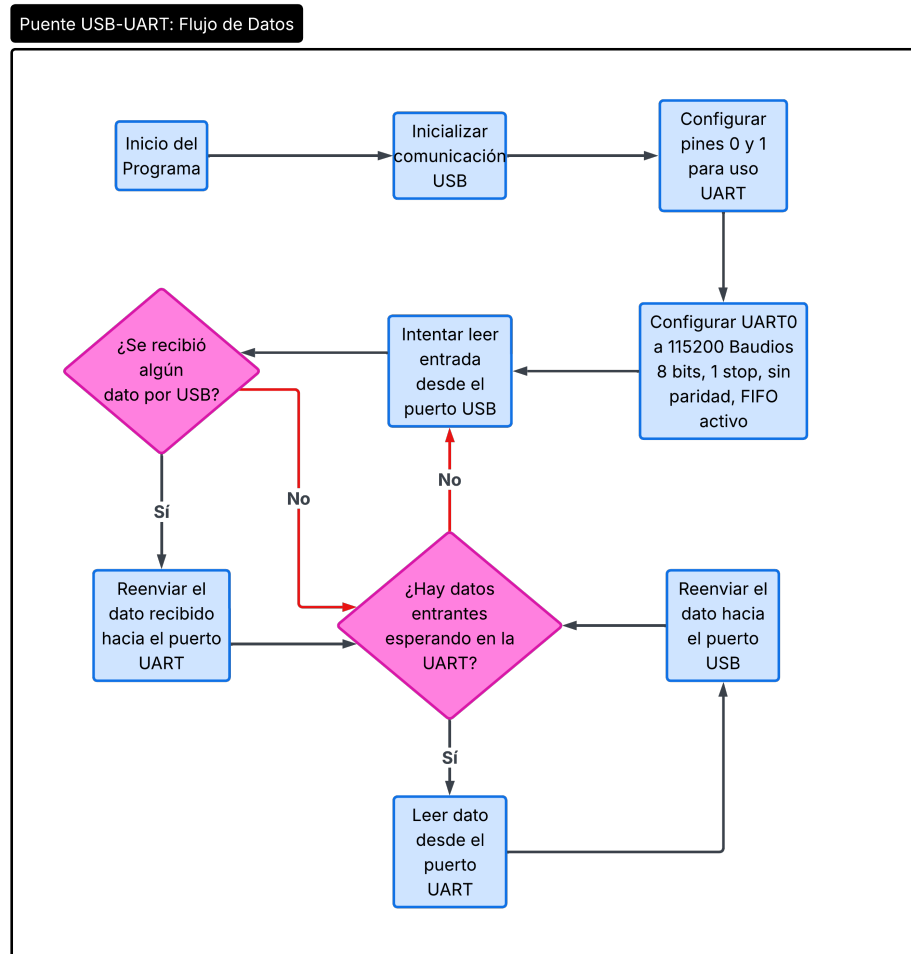


Figura 2. Diagrama de flujo del puente USB–UART que ejecuta la Pico de apoyo.

Desarrollo

El programa de la Pico principal se organiza en tres capas que no se mezclan entre sí. La capa más baja es la rutina de atención a la interrupción del UART, cuya única responsabilidad es mover bytes entre el hardware y dos colas en memoria. La capa intermedia son esas dos colas circulares, una de recepción y otra de transmisión, que desacoplan el ritmo del hardware del ritmo del programa. La capa superior es un bucle de tareas cooperativas que consume la cola de recepción, interpreta comandos y produce texto hacia la cola de transmisión. Ninguna de las tres usa `sleep_ms`, y ninguna tarea se queda esperando a que algo ocurra.

Configuración del enlace y estructuras de datos

El primer bloque define los parámetros del enlace y reserva las estructuras que sostienen todo lo demás. El UART0 trabaja a 115200 baud sobre los pines GP0 y GP1, el LED de estado está en GP16 y los dos pines de traza en GP13 y GP14. La constante

RETARDO_LED_MS es el parámetro que se modifica en la parte 2 de la práctica para inyectar carga artificial en el bucle.

El buffer de recepción se declara con 256 posiciones y dos índices independientes: rx_head, que solo mueve la ISR al escribir, y rx_tail, que solo mueve el bucle al leer. Las tres variables se declaran volatile porque las comparten dos contextos de ejecución distintos —la interrupción y el programa principal— y sin ese calificativo el compilador podría suponer que nadie más las modifica y mantenerlas en un registro, con lo cual el bucle nunca vería los bytes que la ISR acaba de depositar. Junto a ellas se declaran los tres contadores de diagnóstico: overruns, framing_errors y descartados. El buffer de transmisión es simétrico: mismo tamaño, misma disciplina de dos índices.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdint.h>
4 #include <stdbool.h>
5
6 #include "pico/stdlib.h"
7
8 #include "hardware/uart.h"
9 #include "hardware/irq.h"
10 #include "hardware/sync.h"
11 #include "hardware/regs/uart.h"
12
13 // Variables para configurar la UART
14 #define UART_ID uart0
15 #define BAUD_RATE 115200
16
17 #define TX_PIN 0
18 #define RX_PIN 1
19
20 #define LED_PIN 16
21
22 // Trazas para osciloscopio
23 #define TRACE_ISR_PIN 14
24 #define TRACE_LOOP_PIN 13
25
26 // Retardo para el LED
27 #define RETARDO_LED_MS 0
28
29 // Variables para configurar el buffer de recepción
30 #define RX_BUF_SIZE 256
31
32 volatile uint8_t rx_buffer[RX_BUF_SIZE];
33 volatile uint16_t rx_head = 0;
34 volatile uint16_t rx_tail = 0;
35
36 // Variable para contar los caracteres descartados
37 volatile uint32_t descartados = 0;
38
39 // Error contadores
40 volatile uint32_t overruns = 0;
41 volatile uint32_t framing_errors = 0;
42
43 // Variables para configurar el buffer de transmisión
44 #define TX_BUF_SIZE 256
45
46 volatile uint8_t tx_buffer[TX_BUF_SIZE];
47 volatile uint16_t tx_head = 0;
48 volatile uint16_t tx_tail = 0;
```

Figura 3. Parámetros del enlace UART, pines de traza y declaración de los buffers circulares de recepción y transmisión con sus contadores de diagnóstico.

Transmisión por cola: que imprimir no bloquee

Escribir con `printf` sobre un UART es una operación bloqueante: si la FIFO de transmisión está llena, el programa se detiene ahí hasta que se libere un lugar. Una línea de log de cuarenta caracteres cuesta cerca de 3.5 ms de espera, más que el presupuesto completo de recepción del periférico, de modo que imprimir una sola línea puede costar un byte recibido. La solución es aplicar la misma cola circular, pero al revés: quien quiere imprimir deposita los caracteres en `tx_buffer` y regresa de inmediato, y es la ISR la que los va sacando conforme el hardware tenga lugar.

La función `tx_encolar` calcula la siguiente posición, verifica que la cola no esté llena y, solo si logró encolar algo, habilita la interrupción de transmisión. Ese detalle es importante: la IRQ de TX se enciende únicamente cuando hay trabajo pendiente, y se apaga sola cuando la cola queda vacía. Si se dejara permanentemente habilitada, el periférico generaría interrupciones sin descanso pidiendo datos que no existen. La función `tx_encolar_texto` es simplemente el recorrido de una cadena carácter por carácter.

```

1 // Tamaño máximo de la línea de comando (escritura en serial monitor)
2 #define LINE_SIZE 64
3
4 static char linea[LINE_SIZE];
5 static uint8_t n = 0;
6
7 // Configuración LED
8 uint32_t periodo_led_ms = 500;
9 uint32_t ultimo_led = 0;
10
11 bool estado_led = false;
12 bool led_habilitado = true;
13
14 // Configuración log
15 uint32_t ultimo_log = 0;
16
17 // Función para encolar un carácter en el buffer de transmisión
18 static void tx_encolar(uint8_t c) {
19
20     uint16_t siguiente = (tx_head + 1) % TX_BUF_SIZE;
21
22     // Si siguiente == tail, la cola esta llena
23     if (siguiente != tx_tail) {
24
25         tx_buffer[tx_head] = c;
26         tx_head = siguiente;
27
28         // Al habilitar TX, la ISR comenzara a vaciar la cola.
29         uart_set_irq_enables(UART_ID,true,true);
30     }
31 }
32
33 // Función para encolar un texto en el buffer de transmisión
34 static void tx_encolar_texto(const char *texto) {
35
36     // Encolar cada carácter hasta el final del texto
37     while (*texto != '\0') {
38         tx_encolar((uint8_t)*texto);
39         texto++;
40     }
41 }

```

Figura 4. Funciones tx_encolar y tx_encolar_texto, que depositan caracteres en la cola de transmisión y habilitan la interrupción de TX solo cuando hay datos pendientes.

La rutina de atención a la interrupción

La ISR es el punto más delicado del programa y por eso se mantuvo deliberadamente corta: no interpreta nada, no imprime nada y no hace aritmética de punto flotante. Marca el pin de traza, mueve bytes y se va.

La recepción usa while y no if. La diferencia no es cosmética: entre una interrupción y la siguiente pueden haber llegado varios bytes a la FIFO de hardware, y si la rutina sacara uno solo por invocación, los demás quedarían atorados hasta que nuevo tráfico volviera a disparar la interrupción. El lazo vacía la FIFO por completo antes de salir.

Cada lectura se hace sobre el registro UARTDR completo y no sobre los ocho bits de datos, porque ese registro lleva adjuntas las banderas de error del byte que se está leyendo. Se

comprueba primero `UART_UARTDR_FE_BITS`, que indica un error de trama —el receptor no encontró el bit de stop donde lo esperaba—, y después `UART_UARTDR_OE_BITS`, que indica un desbordamiento de la FIFO. Solo entonces se extraen los ocho bits útiles con la máscara `0xFF`. Aquí no se llama a ninguna función de reconocimiento de la interrupción, como sí ocurría con las interrupciones de GPIO: en el UART la bandera se limpia al leer el dato, de modo que leer es reconocer.

Antes de escribir en el buffer circular se verifica que haya lugar. La política elegida cuando la cola está llena es descartar el byte nuevo y llevar la cuenta en descartados. Para una interfaz de comandos es la decisión correcta: un comando recibido a medias es inútil de todas formas, y conservar lo viejo al menos preserva las líneas completas que ya están esperando. Lo que no era aceptable era perder datos en silencio, y por eso todo descarte queda registrado.

El segundo lazo atiende la transmisión: mientras el hardware acepte datos y quede algo en la cola, envía. Al terminar, si la cola quedó vacía, deshabilita la interrupción de TX y deja habilitada únicamente la de RX.

```

1 // ISR para la UART
2 static void on_uart_irq(void) {
3
4 // Traza de ISR
5 gpio_put(TRACE_ISR_PIN, 1);
6
7 // Recepción de datos
8 while (uart_is_readable(UART_ID)) {
9
10 uint32_t dr = uart_get_hw(UART_ID)->dr; // Leer el registro de dato
11 s
12 // Revisar errores asociados al dato recibido
13 if (dr & UART_UARTDR_FE_BITS) {
14 framing_errors++;
15 }
16
17 // Revisar errores de sobrecarga
18 if (dr & UART_UARTDR_OE_BITS) {
19 overruns++;
20 }
21
22 // Extraer el dato recibido (8 bits)
23 uint8_t c = (uint8_t)(dr & 0xFFu);
24
25 // Siguiendo lugar del buffer circular
26 uint16_t siguiente = (rx_head + 1) % RX_BUF_SIZE;
27
28 // Si siguiente == tail, la cola esta llena
29 if (siguiente != rx_tail) {
30 rx_buffer[rx_head] = c;
31 rx_head = siguiente;
32 }
33 // Buffer circular lleno
34 else {
35 descartados++;
36 }
37 }
38
39 // Transmisión de datos
40 while (uart_is_writable(UART_ID) && tx_tail != tx_head) {
41 uart_get_hw(UART_ID)->dr = tx_buffer[tx_tail];
42 tx_tail = (tx_tail + 1) % TX_BUF_SIZE;
43 }
44
45 // Cola TX vacia
46 if (tx_tail == tx_head) {
47 // TX se apaga hasta que haya nuevos datos
48 uart_set_irq_enables(UART_ID, true, false);
49 }
50 gpio_put(TRACE_ISR_PIN, 0);
51 }

```

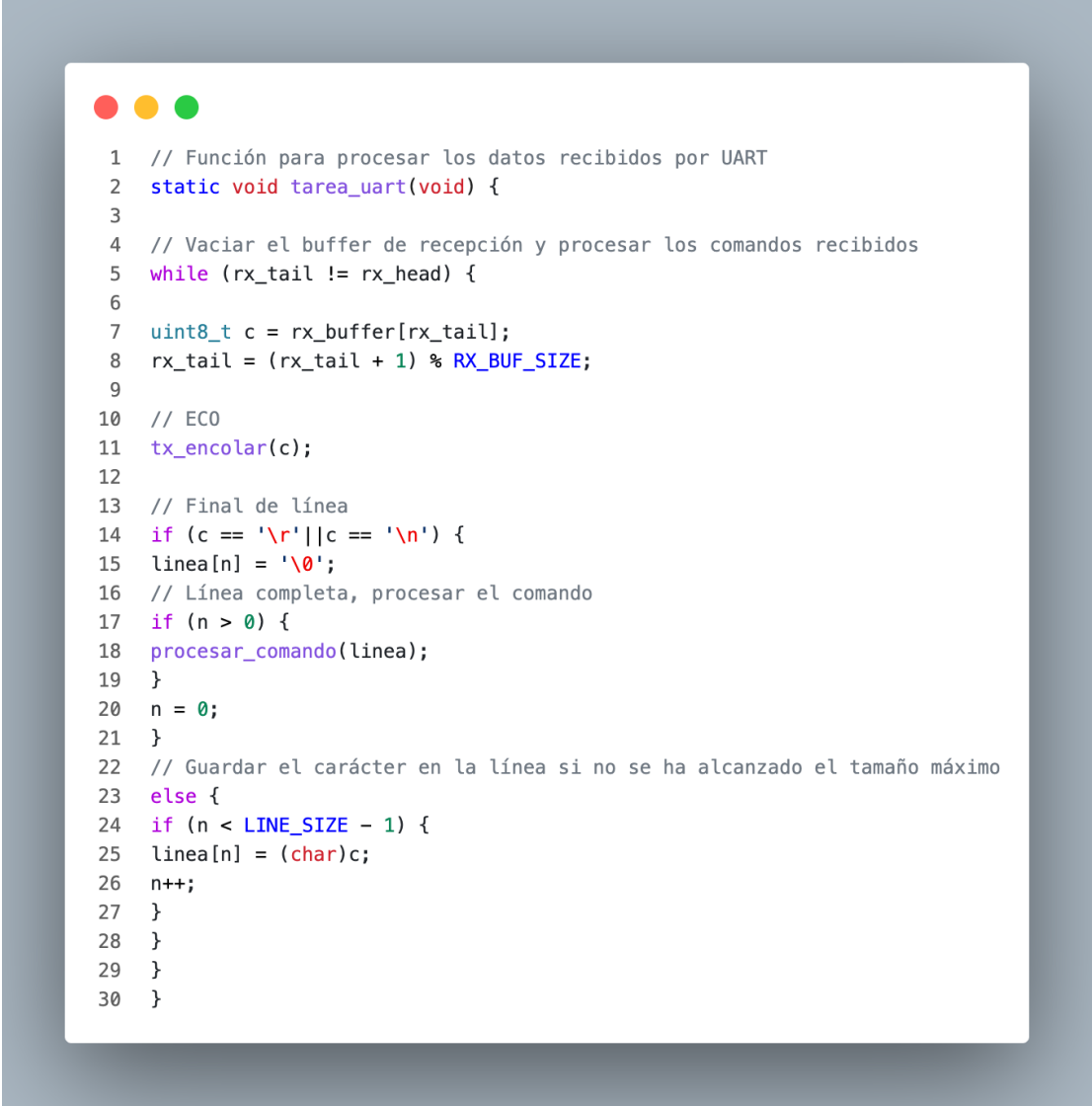
Figura 5. Rutina `on_uart_irq`, que vacía la FIFO de recepción hacia el buffer circular, contabiliza errores de trama y desbordamientos, y alimenta la FIFO de transmisión desde la cola de salida.

La tarea del UART: eco y armado de la línea

Esta tarea se ejecuta en cada vuelta del bucle y es la encargada de consumir lo que la ISR haya dejado. El lazo vacía todo el buffer de una sola vez en lugar de tomar un byte por vuelta, porque si el tráfico llega en ráfagas y el bucle solo consumiera un carácter por iteración, la cola crecería más rápido de lo que se drena.

Cada carácter recibido se devuelve de inmediato a la cola de transmisión. Ese es el eco, y es lo que hace visible lo que se teclea en el monitor serial: el programa no está obligado a devolverlo, pero sin ese reenvío el usuario escribiría a ciegas. Es importante notar que el eco ocurre aquí, en el bucle principal, y no dentro de la ISR.

La línea no está lista hasta que llega el retorno de carro o el salto de línea. Antes de eso, cada carácter se acumula en línea siempre que no se haya alcanzado el tamaño máximo. Al detectar el fin de línea se cierra la cadena con el terminador nulo, se procesa el comando si la línea no estaba vacía, y se reinicia el índice para la siguiente.



```

1 // Función para procesar los datos recibidos por UART
2 static void tarea_uart(void) {
3
4 // Vaciar el buffer de recepción y procesar los comandos recibidos
5 while (rx_tail != rx_head) {
6
7     uint8_t c = rx_buffer[rx_tail];
8     rx_tail = (rx_tail + 1) % RX_BUF_SIZE;
9
10 // ECO
11 tx_encolar(c);
12
13 // Final de línea
14 if (c == '\r' || c == '\n') {
15     linea[n] = '\0';
16     // Línea completa, procesar el comando
17     if (n > 0) {
18         procesar_comando(linea);
19     }
20     n = 0;
21 }
22 // Guardar el carácter en la línea si no se ha alcanzado el tamaño máximo
23 else {
24     if (n < LINE_SIZE - 1) {
25         linea[n] = (char)c;
26         n++;
27     }
28 }
29 }
30 }

```

Figura 6. Tarea `tarea_uart`, que vacía el buffer de recepción, devuelve el eco de cada carácter y arma la línea de comando hasta detectar el fin de línea.

El procesador de comandos

El intérprete cumple el contrato de texto que pide la práctica: un comando por línea, `get` solo consulta y `set` modifica, y toda entrada recibe respuesta, incluso cuando es inválida. El silencio sería indistinguible de un programa colgado.

Los tres primeros comandos se resuelven por comparación exacta de cadenas. En `set led` on se reinicia además la marca de tiempo del LED, para que el primer parpadeo

después de reactivarlo dure el periodo completo y no lo que quedara pendiente del ciclo anterior. El cuarto caso es el único con argumento variable: `sscanf` intenta extraer el número que sigue a `set periodo` y, con el especificador extra `%c`, detecta si venía basura después del valor. Solo se acepta cuando se leyó exactamente un campo y el periodo resultante es mayor que cero; cualquier otra entrada cae en la respuesta de comando desconocido.

```

1 // Función para procesar un comando recibido por UART
2 static void procesar_comando(char *cmd) {
3
4 // Get stats
5 if (strcmp(cmd, "get stats") == 0) {
6 mostrar_stats();
7 }
8
9 // Set led on
10 else if (strcmp(cmd, "set led on") == 0) {
11
12 led_habilitado = true;
13 estado_led = true;
14 gpio_put(LED_PIN, estado_led);
15 ultimo_led = time_us_32();
16 tx_encolar_texto(" ok\r\n");
17
18 }
19
20 // Set led off
21 else if (strcmp(cmd, "set led off") == 0) {
22
23 led_habilitado = false;
24 estado_led = false;
25 gpio_put(LED_PIN, 0);
26 tx_encolar_texto(" ok\r\n");
27 }
28
29 //Set periodo
30 else {
31
32 unsigned int nuevo_periodo;
33 char extra;
34
35 int encontrados = sscanf(
36 cmd,
37 "set periodo %u %c",
38 &nuevo_periodo,
39 &extra
40 );
41
42
43 if (encontrados == 1 && nuevo_periodo > 0) {
44 periodo_led_ms = nuevo_periodo;
45 tx_encolar_texto(" ok\r\n");
46 } else {
47 tx_encolar_texto(" error: comando desconocido\r\n");
48 }
49 }
50 }
51

```

Figura 7. Función `procesar_comando`, que resuelve los cuatro comandos de la interfaz y responde con el mensaje de error ante cualquier entrada no reconocida.

Lectura consistente de los contadores

La respuesta a `get stats` tiene una dificultad que no es evidente: los tres contadores los incrementa la ISR, y el bucle los lee. Si llegara una interrupción justo entre la lectura de overruns y la de descartados, el reporte mezclaría valores de dos instantes distintos. Por

eso se deshabilitan las interrupciones, se copian los tres valores a variables locales y se restaura de inmediato el estado previo con `restore_interrupts` —no se habilitan «a la fuerza», se devuelve exactamente la condición que había—. La sección crítica abarca solo tres asignaciones; el formateo del mensaje, que es la parte costosa, ocurre ya fuera de ella y termina, como todo lo demás, en la cola de transmisión.

```
1 // Función para mostrar estadísticas de errores
2 static void mostrar_stats(void) {
3
4     uint32_t estado_irq = save_and_disable_interrupts(); // Deshabilitar interrupciones para leer variables compartida
5     uint32_t o = overruns; // Guardar el valor de overruns
6     uint32_t f = framing_errors; // Guardar el valor de framing_errors
7     uint32_t d = descartados; // Guardar el valor de descartados
8
9     restore_interrupts(estado_irq); // Restaurar el estado de interrupciones
10
11     char mensaje[128]; // Buffer para el mensaje de estadísticas
12
13     snprintf(
14         mensaje,
15         sizeof(mensaje),
16         "overruns=%lu framing_errors=%lu descartados=%lu\r\n",
17         (unsigned long)o,
18         (unsigned long)f,
19         (unsigned long)d
20     );
21
22     tx_encolar_texto(mensaje);
23 }
```

Figura 8. Función `mostrar_stats`, que copia los tres contadores dentro de una sección crítica mínima antes de formatear la respuesta al comando `get stats`.

Las tareas periódicas y el retardo inyectado

Las dos tareas periódicas siguen el mismo patrón, que es el que reemplaza a `sleep_ms`: en lugar de detenerse a esperar, la tarea pregunta si ya pasó suficiente tiempo desde la última vez y, si no, regresa de inmediato. La comparación se escribe siempre como una resta entre marcas de tiempo, `ahora - ultimo`, y no como una comparación directa entre ellas. La diferencia importa: el contador de microsegundos de 32 bits se desborda cada 71 minutos aproximadamente, y con aritmética sin signo la resta sigue dando el intervalo correcto al cruzar el desbordamiento, mientras que una comparación directa fallaría.

Dentro de `tarea_led` está el bloque que permite el experimento de la parte 2. La directiva `#if RETARDO_LED_MS > 0` hace que la llamada a `busy_wait_ms` exista en el binario solo cuando el retardo es distinto de cero, de modo que la versión sin carga no paga siquiera el costo de evaluar una condición. Se eligió `busy_wait_ms` y no `sleep_ms` de manera

deliberada: se buscaba ocupar el procesador, no cederlo. El retardo se aplica únicamente cuando la tarea efectivamente conmuta el LED, es decir una vez cada periodo, y no en cada vuelta del bucle; esta distinción resultó decisiva al interpretar los resultados.

tarea_log emite cada segundo una línea con la marca de tiempo, el nombre de la tarea, el estado del LED y su periodo vigente. Al enviarse por la cola de transmisión, su costo para el bucle es el de copiar la cadena a memoria, no el de esperar a que los caracteres salgan por el cable.

```

1 //Tarea para controlar el LED
2 static void tarea_led(uint32_t ahora) {
3
4     if (!led_habilitado) {
5         return;
6     }
7
8     if (ahora - ultimo_led >= periodo_led_ms * 1000u)
9         ultimo_led = ahora;
10    estado_led = !estado_led;
11    gpio_put(LED_PIN, estado_led);
12
13
14    // Retardo para el LED
15    #if RETARDO_LED_MS > 0
16
17    busy_wait_ms(RETARDO_LED_MS);
18    #endif
19 }
20 }
21
22 // Tarea para logs periódicos
23 static void tarea_log(uint32_t ahora) {
24
25     if (ahora - ultimo_log >= 1000000u) {
26
27         ultimo_log = ahora;
28         char mensaje[128];
29
30         snprintf(
31             mensaje,
32             sizeof(mensaje),
33             "[%lu us] tarea_log led=%s periodo=%lu ms\r\n",
34             (unsigned long)ahora,
35             led_habilitado ? "on" : "off",
36             (unsigned long)periodo_led_ms
37         );
38
39         // Enviar el mensaje de log por UART
40         tx_encolar_texto(mensaje);
41     }
42 }

```

Figura 9. Tareas tarea_led y tarea_log, con el bloque condicional que inyecta el retardo controlado mediante busy_wait_ms.

Inicialización y bucle principal

La función `main` configura las salidas digitales, asigna la función de UART a GP0 y GP1, e inicializa el periférico. La llamada a `uart_init` devuelve el baud que el hardware realmente consiguió, que se guarda para calcular el error frente al valor nominal y se reporta al arranque. El formato se fija explícitamente en 8N1 y se habilita la FIFO, que es la que proporciona el margen de tolerancia del periférico.

Las interrupciones se activan siguiendo el modelo de los dos candados: primero se registra el manejador y se habilita la línea de interrupción en el controlador con `irq_set_enabled`, y después se habilitan las fuentes dentro del propio periférico con `uart_set_irq_enables`. Nótese que se habilita la recepción pero no la transmisión: esta última permanece apagada hasta que alguien encola el primer carácter.

El bucle principal es deliberadamente plano. Levanta el pin de traza, lee una sola vez el tiempo actual y lo pasa como argumento a las tareas —de modo que las tres razonen sobre el mismo instante—, ejecuta la tarea del UART en cada vuelta, luego la del LED y la del log, y finalmente baja el pin de traza. El ancho del pulso resultante es exactamente la duración de la vuelta, que es la magnitud que gobierna todo el análisis de la parte 2.

```

1  int main(void) {
2
3  // Inicializar como GPIO la salida del LED
4  gpio_init(LED_PIN);
5  gpio_set_dir(LED_PIN, GPIO_OUT);
6  gpio_put(LED_PIN, 0);
7
8  // Inicializar como GPIO las salidas de trazas
9  gpio_init(TRACE_ISR_PIN);
10 gpio_set_dir(TRACE_ISR_PIN, GPIO_OUT);
11 gpio_put(TRACE_ISR_PIN, 0);
12
13
14 gpio_init(TRACE_LOOP_PIN);
15 gpio_set_dir(TRACE_LOOP_PIN, GPIO_OUT);
16 gpio_put(TRACE_LOOP_PIN, 0);
17
18 // Inicializar como UART los pines de transmisión y recepción
19 gpio_set_function(TX_PIN, GPIO_FUNC_UART);
20 gpio_set_function(RX_PIN, GPIO_FUNC_UART);
21
22 // Inicializar la UART con la velocidad BAUD_RATE
23 uint actual_baud = uart_init(UART_ID, BAUD_RATE);
24
25 // Configurar formato de datos: 8 bits, 1 bit de stop, sin paridad
26 uart_set_format(UART_ID, 8, 1, UART_PARITY_NONE);
27
28 // FIFO habilitada
29 uart_set_fifo_enabled(UART_ID, true);
30
31 // Interrupciones de la UART
32 irq_set_exclusive_handler(UART0_IRQ, on_uart_irq);
33 irq_set_enabled(UART0_IRQ, true);
34 uart_set_irq_enabled(UART_ID, true, false);
35
36 // Calcular el error de velocidad de transmisión
37 int32_t diferencia = (int32_t)actual_baud - (int32_t)BAUD_RATE;
38
39 // Calcular el error en partes por millón (ppm)
40 int32_t error_ppm = (int32_t)((int64_t)diferencia * 1000000LL) / BAUD_RATE;
41
42 // Inicializar variables
43 char inicio[128];
44 sprintf(inicio,
45         "UART lista: nominal=%u real=%u error=%ld ppm\r\n",
46         BAUD_RATE,
47         actual_baud,
48         (long)error_ppm);
49
50 // Encolar texto
51 tx_enqueue_texto(inicio);
52
53 // Inicializar marcas de tiempo
54 uint32_t ahora = time_us_32();
55
56 // Bucle principal
57 while (true) {
58     // Cada vuelta
59     tarea_uart();
60
61     // Periódica
62     tarea_led(ahora);
63
64     // Cada segundo
65     tarea_log(ahora);
66
67     // Actualizar tiempo
68     ahora = time_us_32();
69 }
70
71 return 0;
72 }

```

Figura 10. Función main, con la inicialización del UART, el registro de la interrupción, el cálculo del error de baud y el bucle cooperativo de tres tareas.

El puente USB-UART de la Pico de apoyo

El programa de la tarjeta de apoyo no tiene interrupciones ni colas porque no las necesita: es un traductor de un solo sentido por dirección. Inicializa la interfaz USB con `stdio_init_all` y el UART0 con la misma configuración 8N1 que la principal, y después alterna entre dos comprobaciones. La primera lee un carácter del USB sin bloquear, gracias al tiempo de espera de cero microsegundos, y lo reenvía al UART. La segunda vacía todo lo que haya llegado por el UART y lo entrega al USB.

Su velocidad está definida en una sola constante, `BAUD_RATE`, y esa es precisamente la variable que se modifica en la prueba de la parte 2.4: basta cambiar ese número y recargar únicamente esta tarjeta para provocar el desajuste de baudios, sin tocar la Pico principal y, por tanto, sin perder los contadores que ya había acumulado.

```
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3  #include "hardware/uart.h"
4
5
6  // Variables para configurar la UART
7  #define UART_ID uart0
8  #define BAUD_RATE 115200
9
10 #define TX_PIN 0
11 #define RX_PIN 1
12
13
14 int main(void) {
15
16     // Inicialización de la UART y del USB
17     stdio_init_all();
18
19
20     // Configuración de los pines para la UART
21     gpio_set_function(TX_PIN, GPIO_FUNC_UART); // Configura el pin TX como función UART
22     gpio_set_function(RX_PIN, GPIO_FUNC_UART); // Configura el pin RX como función UART
23     uart_init(UART_ID, BAUD_RATE); // Inicializa la UART con la velocidad de baudios especificada
24     uart_set_format(UART_ID, 8, 1, UART_PARITY_NONE); // Configura la UART con 8 bits de datos, 1 bit de parada y sin paridad
25     uart_set_fifo_enabled(UART_ID, true); // Habilita el FIFO de la UART
26
27     while (true) {
28
29         // PRINCIPAL -> USB -> APOYO -> UART -> PC
30         int ch = getchar_timeout_us(0);
31
32         if (ch != PICO_ERROR_TIMEOUT) {
33             uart_putc_raw(UART_ID, (char)ch);
34         }
35
36         // APOYO -> UART -> PC -> APOYO -> USB
37         while (uart_is_readable(UART_ID)) {
38             char c = uart_getc(UART_ID);
39             putchar(c);
40         }
41     }
42     return 0;
43 }
```

Figura 11. Programa completo de la Pico de apoyo, que traslada bytes entre el puerto USB y el UART0 en ambos sentidos sin bloquear.

Generación del tráfico continuo de prueba

Para la parte 2 se necesitaba tráfico de entrada sostenido, imposible de producir tecleando. Se empleó un script de Python con `pyserial` que abre el puerto de la Pico de apoyo y, durante una ventana de treinta segundos medida con `time.monotonic()`, escribe repetidamente una cadena de prueba acumulando en enviados el valor de retorno de cada write. Dentro del mismo lazo se lee y se descarta todo lo que regrese —el eco y las líneas de log—, porque de lo contrario el buffer de entrada del sistema operativo se llenaría y el flujo se frenaría solo, falseando la medición. Al terminar la ventana el script cierra el puerto e imprime el total de bytes entregados, que es el dato que permite verificar que la carga aplicada fue realmente la esperada. El procedimiento se repitió

idéntico para los cuatro valores de retardo, y después de cada corrida se consultó get stats desde el monitor serial.

Resultados

Parte 1 — La consola

1.1 Velocidad real del enlace y su error

Al arrancar, el programa reporta por el enlace la velocidad que uart_init consiguió realmente. La lectura obtenida fue:

```
UART lista: nominal=115200 real=115207 error=60 ppm
```

El baud real difiere del nominal en apenas 7 baud. Expresado como porcentaje:

$$\text{Error \%} = |115207 - 115200| / 115200 \times 100$$

$$\text{Error \%} = 7 / 115200 \times 100 = 0.0061 \%$$

$$\text{En partes por millón: } 7 / 115200 \times 10^6 = 60.76 \text{ ppm} \approx 60 \text{ ppm}$$

El valor de 60 ppm que aparece en pantalla corresponde al truncamiento entero de 60.76 que hace el cálculo del programa. La discrepancia tiene un origen concreto: el baud no se genera de forma arbitraria, sino dividiendo el reloj del periférico. Con el reloj de periféricos del RP2350 a 150 MHz, el divisor ideal sería

$$\text{divisor} = 150\,000\,000 / (16 \times 115200) = 81.3802$$

$$\text{parte entera} = 81 \quad \cdot \quad \text{parte fraccionaria} = 0.3802 \times 64 = 24.33 \rightarrow 24$$

$$\text{divisor efectivo} = 81 + 24/64 = 81.375$$

$$\text{baud real} = 150\,000\,000 / (16 \times 81.375) = 115\,207.37 \text{ baud}$$

El resultado coincide exactamente con el valor que reporta el hardware. El error proviene, entonces, de que el registro fraccionario del divisor tiene solo seis bits de resolución y no puede representar 0.3802 de manera exacta. La regla práctica exige que cada extremo se mantenga dentro del $\pm 3 \%$; con 0.0061 % el enlace opera con un margen cercano a

quinientas veces la tolerancia, por lo que la velocidad quedó descartada como posible fuente de error para el resto de las pruebas.

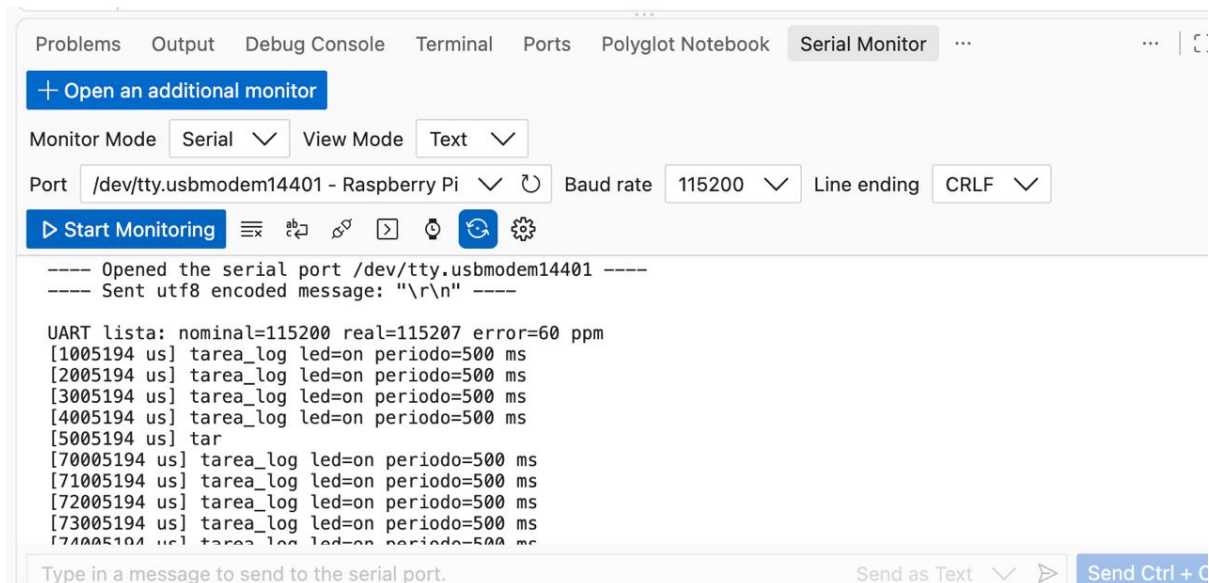


Figura 12. Mensaje de arranque de la Pico principal indicando el baud nominal, el baud real conseguido y el error en partes por millón.

1.3 y 1.7 Eco y respuesta a comandos

La misma captura de la figura 12 confirma que la tarea de log emite una línea por segundo con marca de tiempo, nombre de tarea, estado del LED y periodo vigente, y que las marcas avanzan en incrementos regulares de un millón de microsegundos.

El eco se verificó enviando cadenas que no corresponden a ningún comando. En la figura 13 se observa cómo el texto tecleado regresa a la pantalla seguido del mensaje de error, y cómo las líneas de log siguen apareciendo intercaladas sin interrupción: la consola y la tarea periódica conviven sin estorbarse.

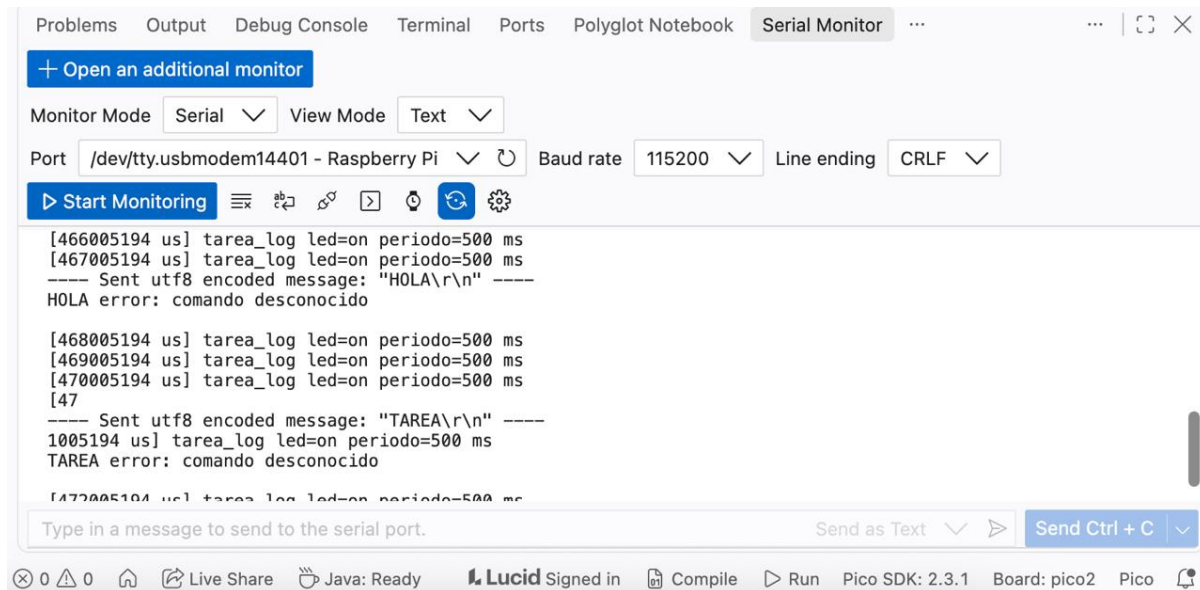


Figura 13. Eco de las cadenas «HOLA» y «TAREA» seguidas de la respuesta de comando desconocido, con las líneas de log apareciendo sin interrupción.

Los cuatro comandos del contrato respondieron conforme a lo especificado. El resumen aparece en la tabla 2 y las capturas correspondientes en las figuras 14 a 17.

Tabla 2. Comandos de la interfaz y respuesta observada en el monitor serial.

Comando enviado	Respuesta observada	Efecto verificado
get stats	overruns=0 framing_errors=0 descartados=0	Reporta los tres contadores
set led on	ok	El log pasa de led=off a led=on
set led off	ok	El log pasa de led=on a led=off
set periodo 1000	ok	El log pasa de periodo=500 a periodo=1000 ms
cualquier otra entrada	error: comando desconocido	Ninguno; el estado se conserva

Conviene señalar un detalle de las capturas: la respuesta aparece pegada al eco del comando, como en `get statsoverruns=0`. No se trata de un defecto del intérprete sino de una consecuencia directa del eco. El programa devuelve carácter por carácter lo que recibe, incluido el retorno de carro final, y a continuación encola su respuesta; el monitor serial muestra ambos flujos en la misma línea porque llegan por el mismo canal. La verificación de que el comando se interpretó correctamente no está en la apariencia del texto, sino en el cambio observable en las líneas de log posteriores.

Problems Output Debug Console Terminal Ports Polyglot Notebook **Serial Monitor** ...

+ Open an additional monitor

Monitor Mode **Serial** View Mode **Text**

Port **/dev/tty.usbmodem14401 - Raspberry Pi** Baud rate **115200** Line ending **CRLF**

Start Monitoring

```
[560005194 us] tarea_log led=on periodo=500 ms
[561005194 us] tarea_log led=on periodo=500 ms
[562005194 us] tarea_log led=on periodo=500 ms
[563005194 us] tarea_log led=on periodo=500 ms
[564005194 us] tarea_log led=on periodo=500 ms
[565005194 us] tarea_log led=on periodo=500 ms
[566005194 us] tarea_log led=on periodo=500 ms
[567005194 us] tarea_log led=on periodo=500 ms
---- Sent utf8 encoded message: "get stats\r\n" ----
get statsoverruns=0 framing_errors=0 descartados=0

[568005194 us] tarea_log led=on periodo=500 ms
[569005194 us] tarea_log led=on periodo=500 ms
---- Closed the serial port /dev/tty.usbmodem14401 ----
```

Type in a message to send to the serial port. Send as Text Send Ctrl + C

0 0 0 Live Share Java: Ready Lucid Signed in Compile Run Pico SDK: 2.3.1 Board: pico2 Pico

Figura 14. Respuesta al comando `get stats` con los tres contadores de diagnóstico en cero.

Problems Output Debug Console Terminal Ports Polyglot Notebook **Serial Monitor** ...

+ Open an additional monitor

Monitor Mode **Serial** View Mode **Text**

Port **/dev/tty.usbmodem14401 - Raspberry Pi** Baud rate **115200** Line ending **CRLF**

Start Monitoring

```
[926005194 us] tarea_log led=off periodo=500 ms
[927005194 us] tarea_log led=off periodo=500 ms
[928005194 us] tarea_log led=off periodo=500 ms
---- Sent utf8 encoded message: "set led on\r\n" ----
set led on ok

[929005194 us] tarea_log led=on periodo=500 ms
[930005194 us] tarea_log led=on periodo=500 ms
[931005194 us] tarea_log led=on periodo=500 ms
[932005194 us] tarea_log led=on periodo=500 ms
[933005194 us] tarea_log led=on periodo=500 ms
[934005194 us] tarea_log led=on periodo=500 ms
[935005194 us] tarea_log led=on periodo=500 ms
[936005194 us] tarea_log led=on periodo=500 ms
```

Type in a message to send to the serial port. Send as Text Send Ctrl + C

0 0 0 Live Share Java: Ready Lucid Signed in Compile Run Pico SDK: 2.3.1 Board: pico2 Pico

Figura 15. Comando `set led on`: el log cambia de `led=off` a `led=on` a partir de la siguiente emisión.

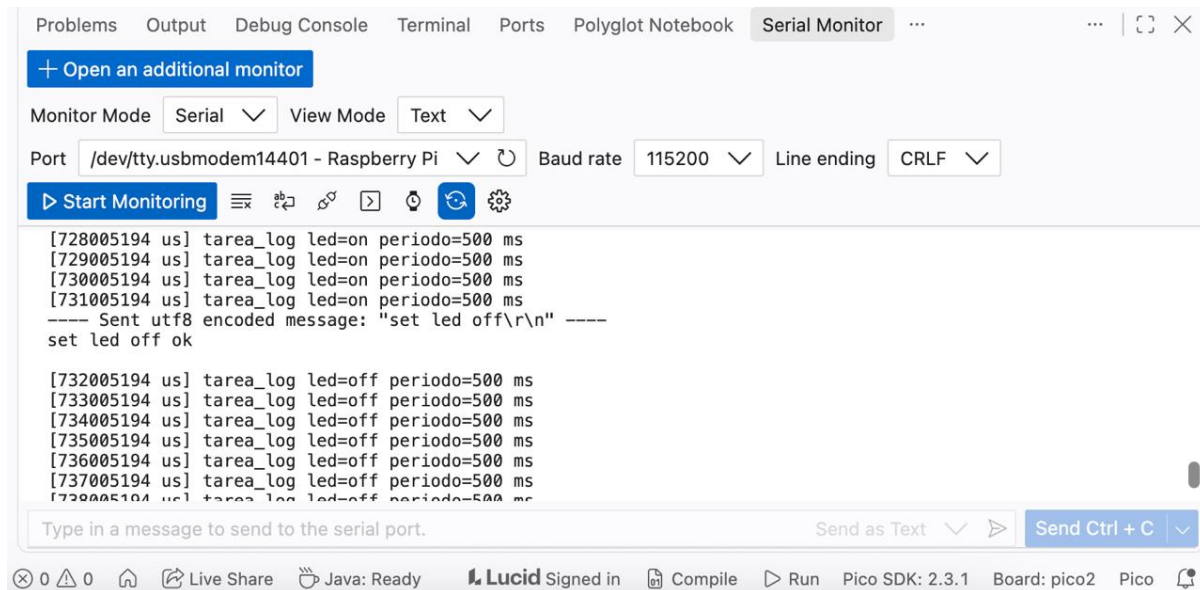


Figura 16. Comando set led off: el log cambia de led=on a led=off y el LED permanece apagado.

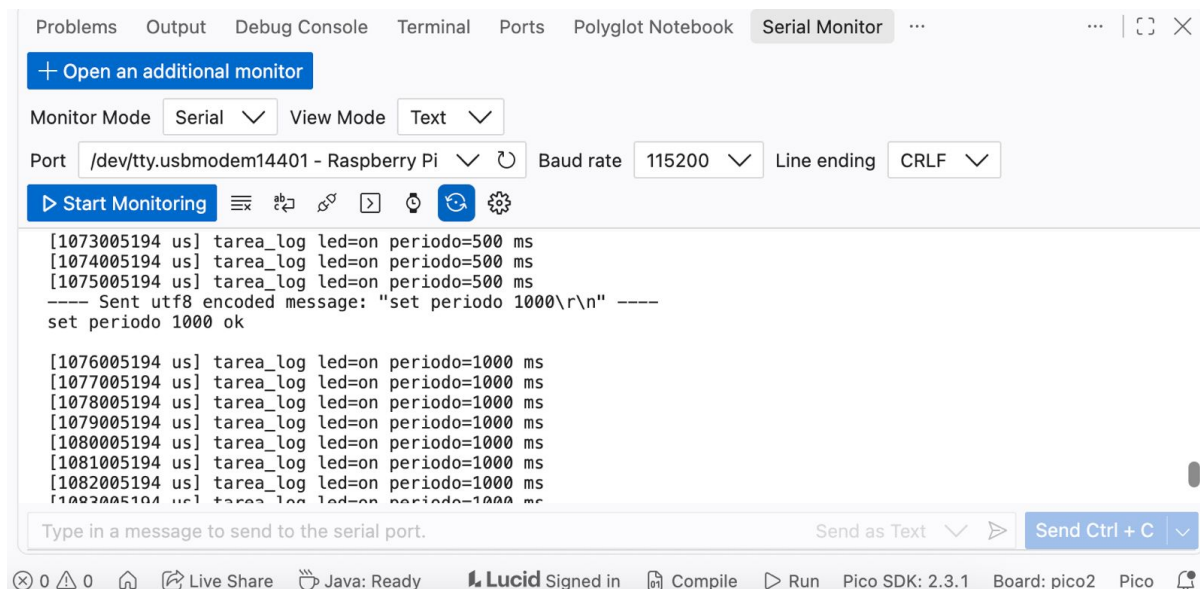


Figura 17. Comando set periodo 1000: el log refleja el nuevo periodo sin necesidad de recompilar ni reiniciar la tarjeta.

Parte 2 — Límite del sistema

2.1 Cálculo del presupuesto de tiempo

En formato 8N1 cada byte viaja acompañado de un bit de start y uno de stop, de modo que transmitir ocho bits de información cuesta diez bits en el cable. A 115200 baud:

$$\text{Duración de un bit} = 1 / 115\,200 = 8.6806 \mu\text{s}$$

$$\text{Duración de un byte} = 10 \times 8.6806 \mu\text{s} = 86.806 \mu\text{s}$$

$$\text{Tasa efectiva} = 115\,200 / 10 = 11\,520 \text{ bytes/s}$$

La FIFO de recepción del periférico PL011 tiene 32 posiciones. Ese buzón es el que determina cuánto tiempo puede transcurrir sin que nadie lo atienda antes de que un byte nuevo se pierda:

$$t_{FIFO} = 32 \times 86.806 \mu s = 2\,777.8 \mu s = 2.78 \text{ ms}$$

Ese es el presupuesto que se maneja en clase y el que se pide calcular: **la vuelta del bucle no debe durar más de 2.78 ms**. Sin embargo, esa cota supone un diseño en el que es el propio bucle quien saca los bytes del hardware. En esta implementación la recepción está delegada a la interrupción, y eso agrega un segundo presupuesto, más generoso, que corresponde al buffer circular de software:

$$\text{Capacidad útil del buffer} = 256 - 1 = 255 \text{ bytes}$$

$$t_{buffer} = 255 \times 86.806 \mu s = 22\,135 \mu s = 22.1 \text{ ms}$$

La capacidad útil es de 255 y no de 256 posiciones porque la condición `head == tail` se reserva para señalar «cola vacía»; si se permitiera llenar la última posición, esa condición se volvería ambigua. Se tienen entonces dos límites distintos que conviene no confundir: **2.78 ms es cuánto puede tardar la ISR en ser atendida** antes de que el hardware pierda un byte, y **22.1 ms es cuánto puede tardar el bucle principal** antes de que el software tenga que descartar. El primero produce overruns; el segundo, descartados.

2.2 Medición del umbral

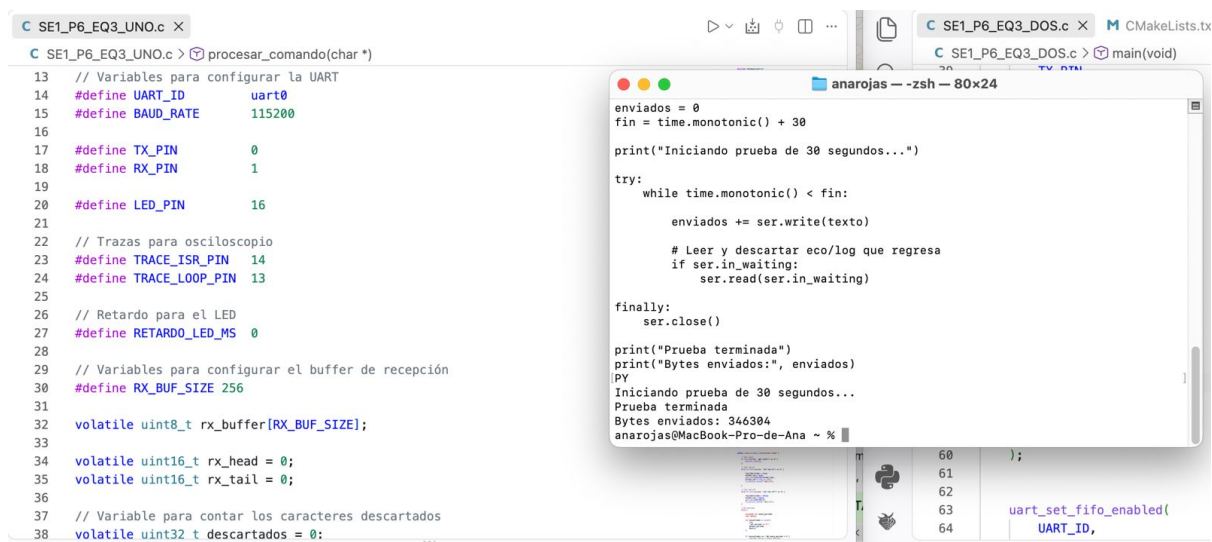
Se compiló el programa con cuatro valores de `RETARDO_LED_MS` y para cada uno se aplicó tráfico continuo durante treinta segundos con el script descrito en el desarrollo, consultando `get_stats` al finalizar cada corrida. La columna «vuelta medida en la traza» que contempla el guion de la práctica no se incluye porque el montaje no dispuso de osciloscopio. Los resultados se presentan en la tabla 3.

Tabla 3. Contadores de diagnóstico tras treinta segundos de tráfico continuo para cada valor de retardo inyectado.

Retardo inyectado	Bytes enviados en 30 s	Overruns	Framing errors	Descartados
0 ms	346 304	0	0	0
1 ms	346 752	0	0	0
3 ms	346 496	0	0	0
10 ms	346 752	0	0	0

Lo primero que debe verificarse es que la carga aplicada fue realmente la esperada. El máximo teórico en una ventana de treinta segundos es de 345 621 bytes al baud real medido, y las cuatro corridas quedaron entre 346 304 y 346 752, es decir alrededor del 100.2 %. El ligero exceso se explica porque write contabiliza los bytes entregados al buffer del sistema operativo y no los que ya salieron por el cable, y porque la última escritura se completa después de que vence la ventana de treinta segundos. La conclusión relevante es que el enlace estuvo saturado durante toda la prueba: no hubo tiempo muerto que pudiera enmascarar una pérdida.

El segundo hecho es que **en ninguno de los cuatro casos se perdió un solo byte**. El umbral de pérdida no se alcanzó dentro del intervalo de retardos que pide la práctica.



The image shows a code editor with a C file named `SE1_P6_EQ3_UNO.c` and a terminal window running the program. The code defines UART parameters and a 30-second test loop. The terminal output shows the test starting, sending data, and finishing with a total of 346304 bytes sent.

```

C SE1_P6_EQ3_UNO.c
C SE1_P6_EQ3_UNO.c > procesar_comando(char *)
13 // Variables para configurar la UART
14 #define UART_ID      uart0
15 #define BAUD_RATE    115200
16
17 #define TX_PIN       0
18 #define RX_PIN       1
19
20 #define LED_PIN      16
21
22 // Trazas para osciloscopio
23 #define TRACE_ISR_PIN 14
24 #define TRACE_LOOP_PIN 13
25
26 // Retardo para el LED
27 #define RETARDO_LED_MS 0
28
29 // Variables para configurar el buffer de recepción
30 #define RX_BUF_SIZE 256
31
32 volatile uint8_t rx_buffer[RX_BUF_SIZE];
33
34 volatile uint16_t rx_head = 0;
35 volatile uint16_t rx_tail = 0;
36
37 // Variable para contar los caracteres descartados
38 volatile uint32_t descartados = 0;

```

```

anarojas -- zsh -- 80x24
enviados = 0
fin = time.monotonic() + 30
print("Iniciando prueba de 30 segundos...")
try:
    while time.monotonic() < fin:
        enviados += ser.write(texto)
        # Leer y descartar eco/log que regresa
        if ser.in_waiting:
            ser.read(ser.in_waiting)
    finally:
        ser.close()
print("Prueba terminada")
print("Bytes enviados:", enviados)
py
Iniciando prueba de 30 segundos...
Prueba terminada
Bytes enviados: 346304
anarojas@MacBook-Pro-de-Ana ~ %

```

Figura 18. Retardo de 0 ms: el script reporta 346 304 bytes entregados durante la ventana de treinta segundos.

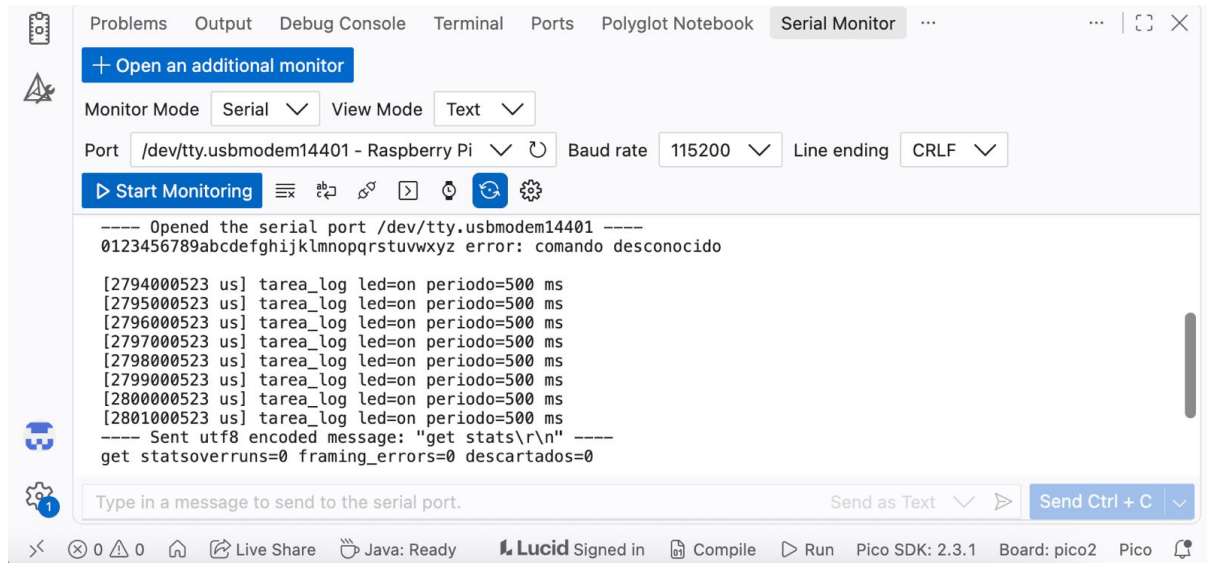


Figura 19. Estadísticas tras la corrida con 0 ms de retardo, con los tres contadores en cero.

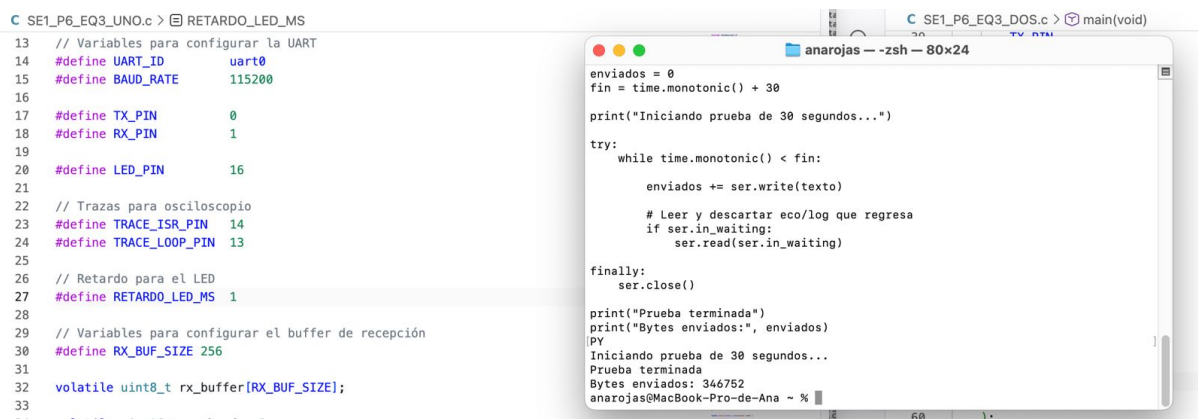


Figura 20. Retardo de 1 ms: el script reporta 346 752 bytes entregados durante la ventana de treinta segundos.

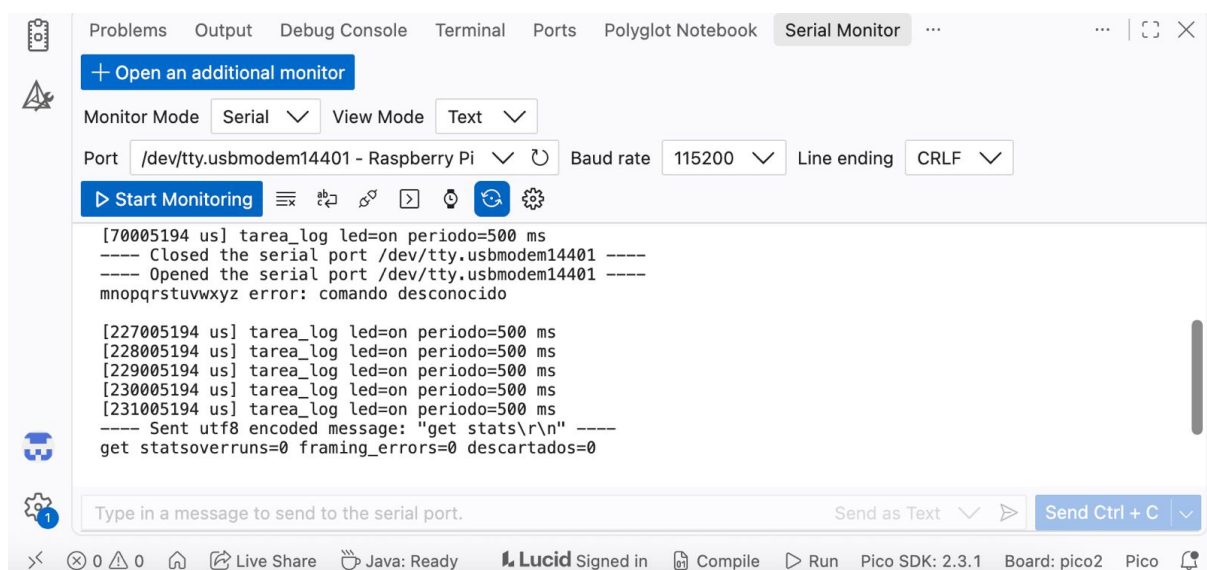


Figura 21. Estadísticas tras la corrida con 1 ms de retardo, sin overruns ni bytes descartados.

```
C SE1_P6_EQ3_UNO.c > RETARDO_LED_MS
13 // Variables para configurar la UART
14 #define UART_ID      uart0
15 #define BAUD_RATE     115200
16
17 #define TX_PIN        0
18 #define RX_PIN        1
19
20 #define LED_PIN        16
21
22 // Trazas para osciloscopio
23 #define TRACE_ISR_PIN  14
24 #define TRACE_LOOP_PIN 13
25
26 // Retardo para el LED
27 #define RETARDO_LED_MS 3
28
29 // Variables para configurar el buffer de recepción
30 #define RX_BUF_SIZE    256
31
32 volatile uint8_t rx_buffer[RX_BUF_SIZE];
33
```

Figura 22. Retardo de 3 ms: el script reporta 346 496 bytes entregados, con el valor de RETARDO_LED_MS visible en el editor.

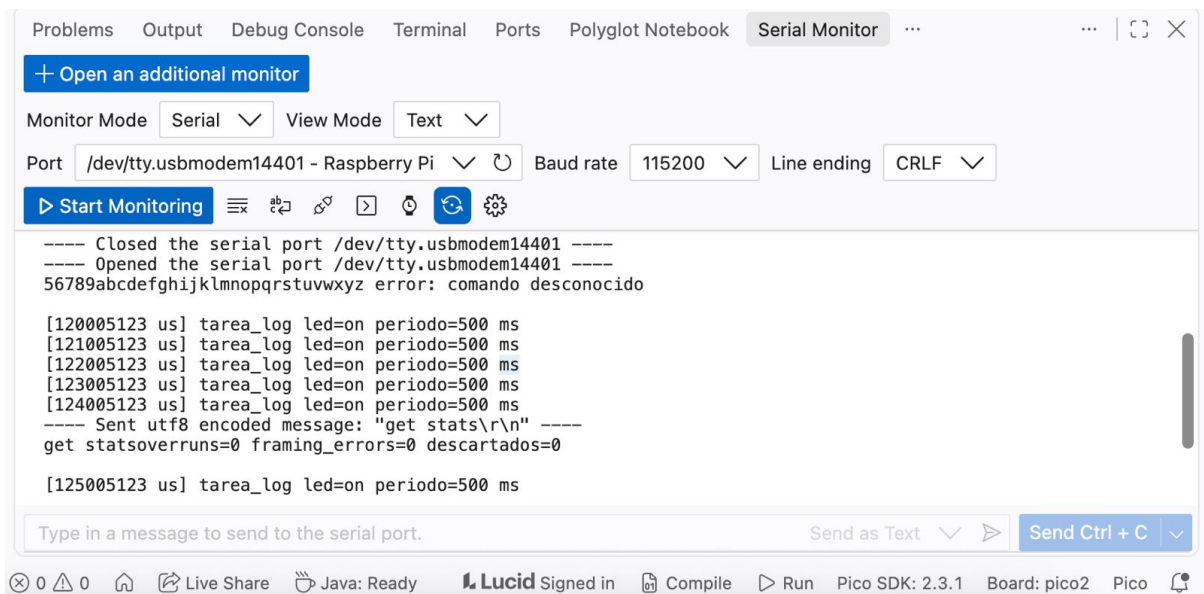


Figura 23. Estadísticas tras la corrida con 3 ms de retardo, superado ya el presupuesto teórico de la FIFO y aún sin pérdidas.

```
C SE1_P6_EQ3_UNO.c > RETARDO_LED_MS
13 // Variables para configurar la UART
14 #define UART_ID      uart0
15 #define BAUD_RATE     115200
16
17 #define TX_PIN        0
18 #define RX_PIN        1
19
20 #define LED_PIN        16
21
22 // Trazas para osciloscopio
23 #define TRACE_ISR_PIN  14
24 #define TRACE_LOOP_PIN 13
25
26 // Retardo para el LED
27 #define RETARDO_LED_MS 10
28
29 // Variables para configurar el buffer de recepci3n
30 #define RX_BUF_SIZE 256
31
32 volatile uint8_t rx_buffer[RX_BUF_SIZE];
33
```

Figura 24. Retardo de 10 ms: el script reporta 346 752 bytes entregados durante la ventana de treinta segundos.

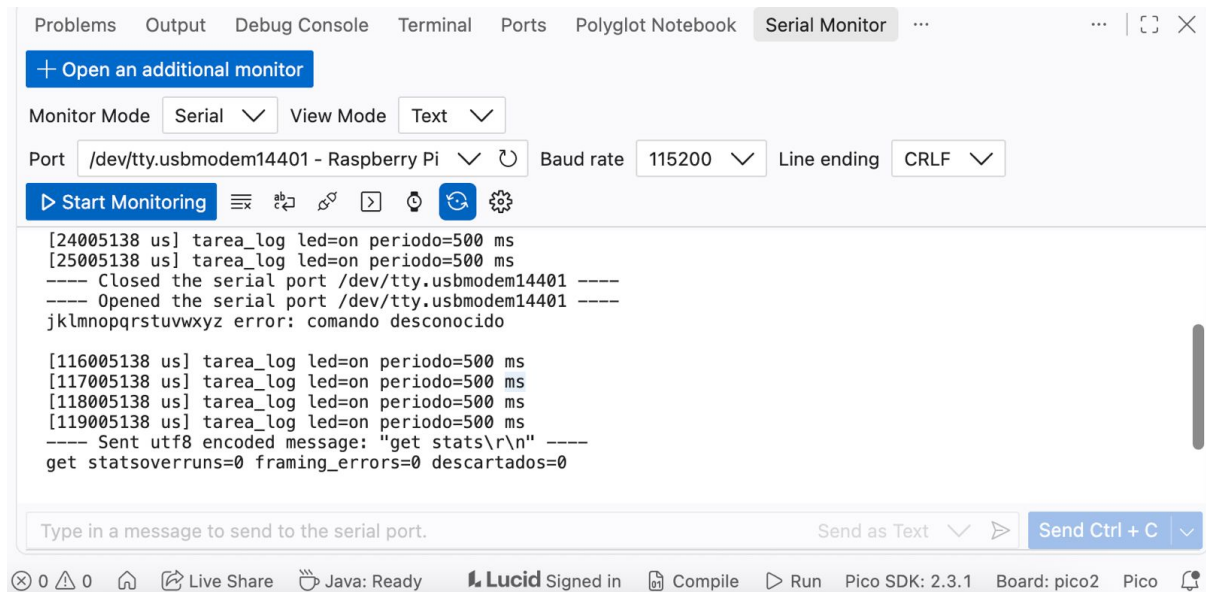


Figura 25. Estadísticas tras la corrida con 10 ms de retardo, el caso más exigente probado, con los tres contadores todavía en cero.

2.3 Comparación entre el umbral calculado y el medido

El cálculo de la sección 2.1 anticipaba que una vuelta del bucle superior a 2.78 ms debía producir desbordamientos. La medición muestra que con 3 ms de retardo —ya por encima de esa cota— y con 10 ms —más de tres veces la cota— los contadores siguen en cero. La discrepancia es real y tiene tres causas que se suman.

Primera: la recepción no depende del bucle. El presupuesto de 2.78 ms describe un diseño por sondeo, en el que el bucle principal es quien vacía la FIFO. Aquí esa responsabilidad está en la ISR, y `busy_wait_ms` no deshabilita las interrupciones: mientras `tarea_led` consume sus 10 ms girando en vacío, la interrupción del UART0 sigue disparándose y vaciando la FIFO hacia el buffer circular. La FIFO de hardware nunca llega a acumular más de unos pocos bytes, y por eso overruns permanece en cero. Para provocar un desbordamiento de hardware habría que impedir que la ISR se ejecute —deshabilitando las interrupciones o deteniendo el procesador con un punto de interrupción— durante más de 2.78 ms.

Segunda: el límite efectivo lo impone el buffer de software. Desplazada la recepción a la interrupción, la cota que gobierna la pérdida ya no es la de la FIFO sino la del buffer circular: 22.1 ms. El caso más exigente probado, 10 ms, representa menos de la mitad de ese margen. Traducido a ocupación del buffer:

Bloqueo de 1 ms → 1 000 μ s / 86.806 μ s $\approx$ 11.5 bytes acumulados

Bloqueo de 3 ms → 3 000 μ s / 86.806 μ s $\approx$ 34.6 bytes acumulados

Bloqueo de 10 ms → 10 000 μ s / 86.806 μ s $\approx$ 115.2 bytes acumulados

Capacidad útil del buffer: 255 bytes → ocupación máxima $\approx$ 45 %

Tercera: el retardo no se aplica en cada vuelta. El bloque `busy_wait_ms` está dentro de la condición que conmuta el LED, de modo que se ejecuta una sola vez cada 500 ms y no en cada iteración. Durante los casi quinientos milisegundos restantes el bucle corre a máxima velocidad y drena el buffer por completo. La ocupación calculada arriba es, por tanto, un pico aislado que se disipa de inmediato, no un nivel sostenido.

De este análisis se desprende una predicción verificable: el umbral de descartados para esta implementación debería situarse alrededor de un retardo inyectado de **22 ms**, mientras que el de overruns no aparecería por más que se aumentara el retardo, porque no depende de la duración de la vuelta sino de que la interrupción quede inhibida. Los resultados de la tabla 3 no contradicen el cálculo de la sección 2.1: confirman que ese cálculo describe un presupuesto distinto del que gobierna este diseño, y la diferencia entre ambos —2.78 ms contra 22.1 ms, casi un factor de ocho— es exactamente la ganancia que aporta haber movido la recepción a la interrupción.

2.4 Error de trama por desajuste de velocidad

Para la última prueba se modificó la constante `BAUD_RATE` de la Pico de apoyo a 57600 y se recargó únicamente esa tarjeta, dejando la principal en 115200. La Pico principal no se reinició de forma deliberada, para conservar los contadores acumulados durante el desajuste.

El efecto en pantalla fue inmediato: el texto dejó de ser legible y apareció en su lugar una secuencia de caracteres sin sentido, del tipo `H??C??L?`, repetida con cierta regularidad. El comando `get_stats` enviado durante el desajuste tampoco pudo interpretarse, porque viajó por el mismo enlace deformado.

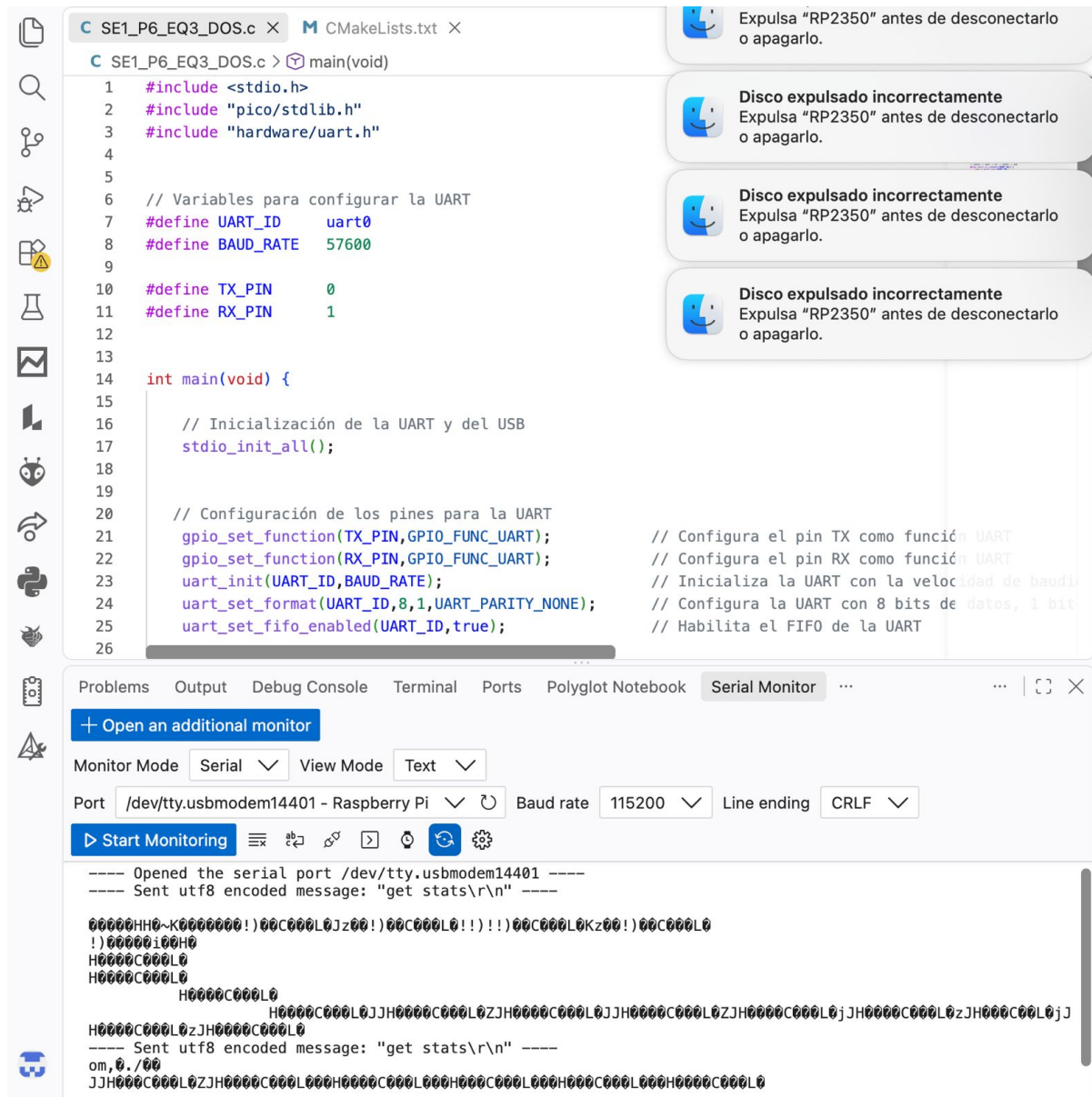


Figura 26. Puente configurado a 57600 baud mientras la Pico principal permanece en 115200, con la salida ilegible que resulta en el monitor serial.

Tras restaurar el puente a 115200 y recargar únicamente esa tarjeta, la comunicación volvió a la normalidad y get_stats reportó el saldo de la prueba:

overruns=0 framing errors=21 descartados=0

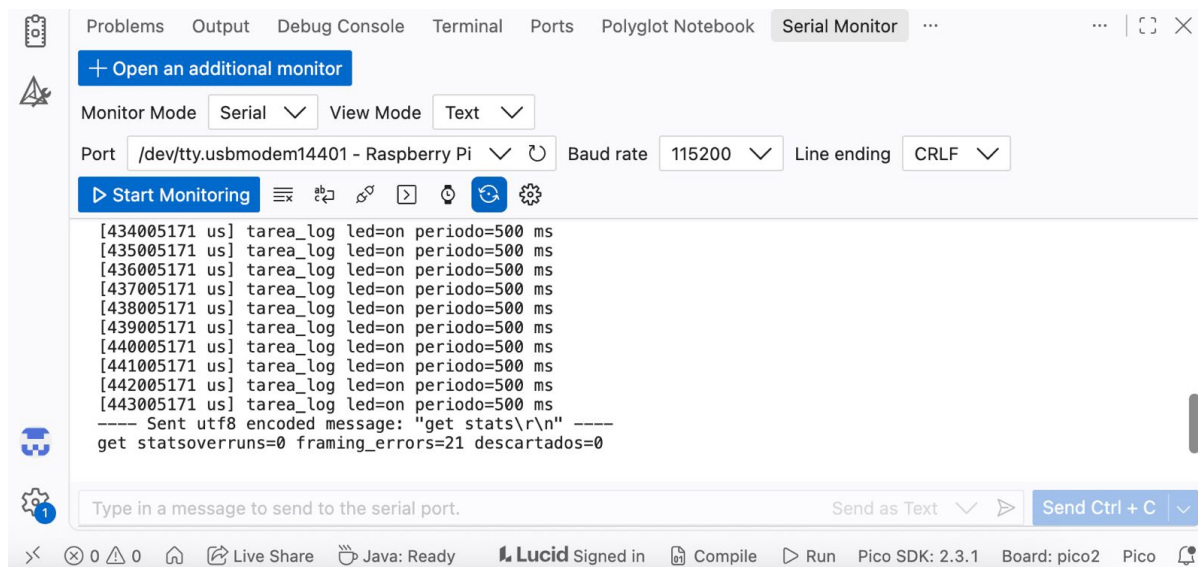


Figura 27. Estadísticas tras restaurar el puente a 115200, con los 21 errores de trama acumulados durante el desajuste y los demás contadores en cero.

La magnitud del desajuste explica el resultado. El receptor muestrea cada bit suponiendo una duración de $8.68 \mu\text{s}$, mientras que el emisor los produce con una duración de $17.36 \mu\text{s}$: un error del 100 %, muy por encima del $\pm 3 \%$ que tolera el protocolo. Cada bit transmitido es interpretado como dos, y cuando el receptor llega a la posición donde espera el bit de stop, la línea no siempre está en alto. Solo en esos casos el hardware levanta la bandera FE del registro UARTDR y el contador se incrementa; en los demás, el byte pasa la verificación de trama pero entrega un valor equivocado, y eso es precisamente lo que produce la basura en pantalla. Los 21 errores contados son, por tanto, un subconjunto de los bytes corrompidos, no su total.

Por qué este error es distinto de un overrun. La diferencia es de naturaleza, no de grado, y la propia medición la demuestra:

Tabla 4. Comparación entre los tres mecanismos de pérdida que registra el programa.

Contador	Qué significa	De quién es la culpa	Cómo se corrige
framing_errors	El byte llegó deformado en el tiempo: el bit de stop no estaba donde se esperaba	Del enlace físico: velocidades distintas, falta de tierra común o ruido	Igualando la velocidad en ambos extremos
overruns	El byte llegó perfecto, pero la FIFO del hardware estaba llena	Del programa: nadie vació la FIFO dentro de los 2.78 ms disponibles	Atendiendo la recepción por interrupción o acortando la vuelta
descartados	El byte llegó a la ISR, pero el buffer circular de software estaba lleno	Del bucle principal: tardó más de 22.1 ms en consumir la cola	Agrandando el buffer o acelerando el consumo

Que overruns se haya mantenido en cero durante toda la prueba confirma que el hardware alcanzó a atender cada byte que llegó, y que descartados también permaneciera en cero confirma que el buffer de software nunca se llenó. Lo único que falló fue la interpretación temporal de cada byte. Un error de trama no se resuelve haciendo el programa más rápido: por mucho que se optimice el código, si los dos extremos no están de acuerdo en la velocidad, el receptor seguirá leyendo en los instantes equivocados. La prueba resultó especialmente valiosa porque aisló los tres mecanismos: permitió observar un modo de falla con los otros dos contadores en cero, demostrando que miden fenómenos independientes.

Evidencias

El montaje físico se armó sobre dos protoboards, con la Pico principal en la superior —conectada al LED de estado y a los pines de traza— y la Pico de apoyo en la inferior, enlazada por USB a la computadora. En la figura 28 el LED verde aparece encendido, correspondiente a un instante del ciclo de parpadeo de la tarea periódica. Los cables rojo y negro llevan la alimentación y la referencia común de tierra entre ambas tarjetas, indispensable para que el enlace serial funcione.

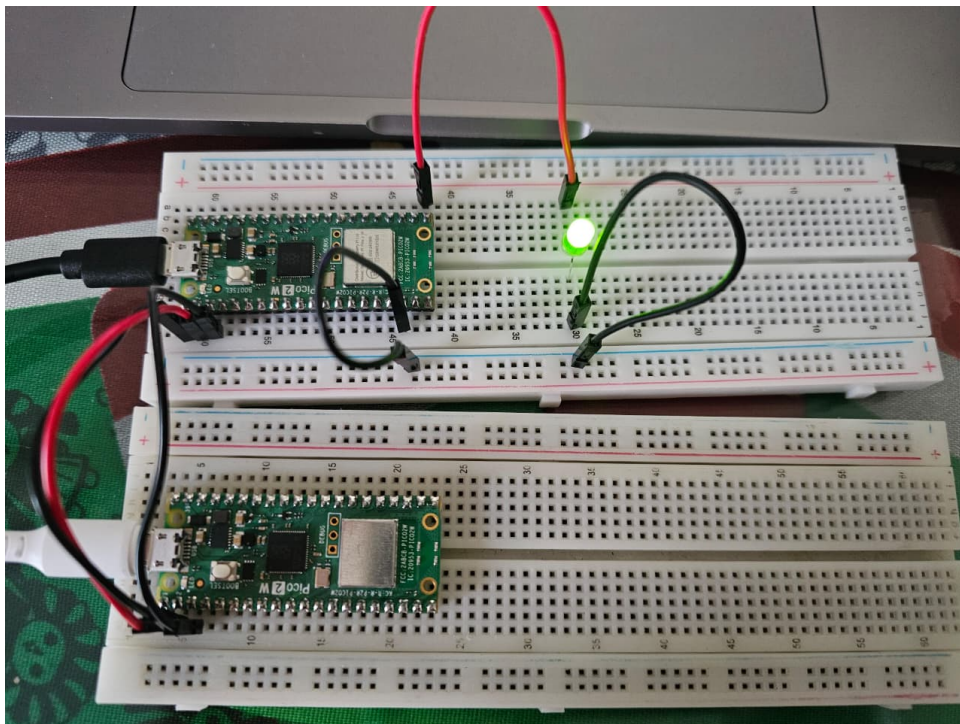


Figura 28. Montaje físico con la Pico principal y el LED de estado en la protoboard superior y la Pico de apoyo conectada por USB en la inferior.

La figura 29 muestra con mayor detalle la salida ilegible que produjo el desajuste de baudios de la sección 2.4. El patrón no es aleatorio: se repiten secuencias reconocibles porque cada byte del mensaje original, muestreado al doble de la velocidad correcta, se traduce siempre al mismo valor equivocado.

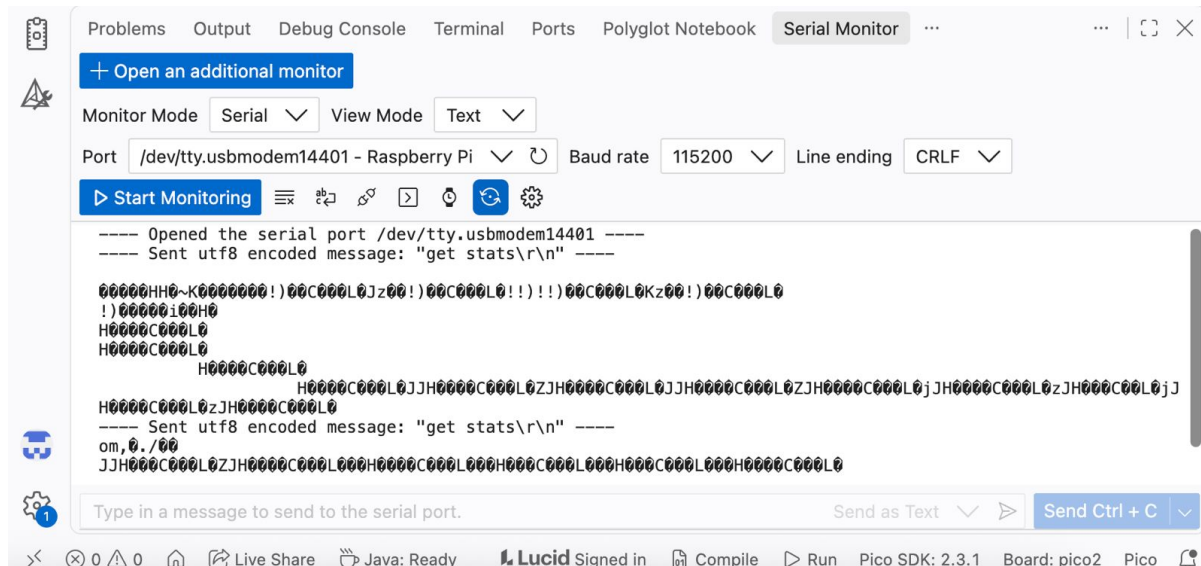


Figura 29. Detalle de la salida en el monitor serial durante el desajuste de baudios, con patrones que se repiten porque cada byte se corrompe de forma determinista.

El código fuente completo de ambas tarjetas, con sus archivos de configuración de CMake, se entrega en el archivo comprimido que acompaña a este reporte, en las carpetas SE1_P6_EQ3_UNO —programa de la Pico principal— y SE1_P6_EQ3_D0S —puente USB-UART de la Pico de apoyo—.

Conclusiones

La práctica demostró que atender comunicación serial y tareas periódicas al mismo tiempo no es cuestión de escribir código más rápido, sino de repartir correctamente las responsabilidades entre la interrupción y el bucle principal. El programa cumplió los siete requisitos de la parte 1: el enlace quedó a 115200 8N1 con un error de 0.0061 %, la recepción se resolvió por interrupción con una ISR que únicamente mueve bytes, el eco se produce desde el bucle, los descartes se contabilizan, la transmisión se hace por cola para que imprimir no bloquee, las tres tareas conviven sin un solo `sleep_ms` y los cuatro comandos responden conforme al contrato establecido.

El resultado más instructivo fue el de la parte 2, y no por lo que se midió sino por lo que no apareció. Se esperaba encontrar el punto de pérdida en algún lugar cercano a los 2.78 ms que marca la FIFO del periférico, y en cambio los contadores siguieron en cero incluso con un retardo de 10 ms. Analizar esa discrepancia fue más valioso que haberla confirmado: obligó a distinguir dos presupuestos que la fórmula inicial confunde en uno solo. Los 2.78 ms limitan cuánto puede tardar la *interrupción* en ser atendida; los 22.1 ms del buffer circular limitan cuánto puede tardar el *bucle* en consumir la cola. Mientras las interrupciones permanezcan habilitadas —y `busy_wait_ms` no las inhibe—, es el segundo límite el que gobierna, y por eso el diseño toleró casi ocho veces lo que predecía el cálculo de clase.

Ese factor de ocho es, en realidad, la medida del beneficio de haber movido la recepción a la ISR. Un diseño por sondeo con la misma carga habría empezado a perder datos alrededor de los 3 ms de retardo, y cualquier impresión directa con `printf` —3.5 ms para una línea de cuarenta caracteres— habría bastado por sí sola para rebasar el presupuesto. La combinación de recepción por interrupción, transmisión por cola y tareas que nunca esperan es la que permite que el sistema siga siendo confiable bajo una carga que lo haría fallar en cualquier otra organización.

La prueba de la sección 2.4 cerró el cuadro al exhibir un modo de falla que ninguna cantidad de optimización habría evitado. Obtener 21 errores de trama con overruns y descartados en cero probó experimentalmente que los tres contadores miden fenómenos independientes: el primero pertenece al enlace físico y a la velocidad acordada; los otros dos, a la capacidad del programa de mantenerse al día. Ese es el valor práctico de instrumentar el código con contadores separados en lugar de un único indicador de error: cuando algo falla, el diagnóstico apunta directamente al lugar correcto.

La limitación principal del trabajo fue la ausencia de osciloscopio, que impidió medir la duración real de la vuelta del bucle y el tiempo de servicio de la ISR sobre los pines de traza GP13 y GP14, previstos en el código. Con esa medición habría sido posible contrastar el retardo inyectado contra la duración efectiva de la vuelta y confirmar directamente que la ISR sigue ejecutándose durante los bloqueos del bucle, que en este reporte se sostiene por razonamiento y por la evidencia indirecta de los contadores.

Como continuación natural quedan dos experimentos: barrer el retardo hasta los 22 ms para localizar el umbral de descartados predicho en la sección 2.3, y forzar un overrun real deshabilitando las interrupciones durante más de 2.78 ms, para comprobar que el contador responde al mecanismo que se le atribuye.

Uso de IA

Herramienta	Uso específico
ChatGPT	Apoyo en la escritura del código y en la compilación desde la terminal para las pruebas del retardo inyectado en la tarea del LED.
GitHub Copilot	Comentado del código fuente de ambas tarjetas.
Gemini	Generación de los códigos en sintaxis Mermaid que sirvieron de base para los diagramas.
Lucid AI	Elaboración de los diagramas de flujo presentados en las figuras 1 y 2.
Claude	Apoyo en la redacción y organización del documento de la práctica.

Referencias

- FreeCodeCamp. 2020. *Finite State Machine Explained*. Recuperado de: <https://www.freecodecamp.org/news/finite-state-machines/>
- Llamas, L. s.f. *Qué son y cómo usar los GPIO en Raspberry Pi*. Recuperado de: <https://www.luisllamas.es/como-usar-gpio-raspberry-pi/>
- Newbiely, s.f. *Raspberry Pi Pico – Button – Debounce*. Recuperado de: <https://newbiely.com/tutorials/raspberry-pico/raspberry-pi-pico-button-debounce>
- Raspberry Pi, 2023. *Use Pico's USB port as a UART for host communication*. Recuperado de: <https://forums.raspberrypi.com/viewtopic.php?t=351506>
- Zaczynsky, B. 2025. *Bitwise Operators in Python*. Recuperado de: <https://realpython.com/python-bitwise-operators/>